

UNIVERSIDAD AUTÓNOMA “JUAN MISAEL SARACHO”

FACULTAD DE CIENCIAS Y TECNOLOGÍA · DIRECCIÓN DE POSGRADO

DIPLOMADO EN DESARROLLO WEB Y APLICACIONES MÓVILES · 2026

TRABAJO FINAL — DOCUMENTO EN AVANCE

Módulo 4 · Entregable E3 · Checkpoint técnico · Semana 3

Campo	Dato
Postulante	Jorge Luis Gutierrez Rivera
Correo electrónico	e73700@uajms.edu.bo
Título provisional del trabajo	Max Pizzapp: sistema web y móvil de gestión de pedidos para recepción y cocina de Max's Pizzas en Villa Fátima
Línea de investigación	Desarrollo de aplicaciones web
Tipo de solución	web y móvil (aplicación web para recepción; APK Android para cocina)
Organización o ámbito beneficiario	Pizzería Max's Pizzas — local central de la zona Villa Fátima, Tarija
Grupo de tutoría	Grupo 1 (lunes, 20:00–21:00)
Decisión de la tutoría T1	Aprobado con recorte
Alcance acordado en T1	El planteamiento inicial abarcaba la operación completa de los tres locales de Max's Pizzas —central, Villa Avaroa y Tabladita—, con cuatro roles, control de inventario, cobro, cierre de caja y reportes consolidados. En la tutoría se acordó recortarlo al tramo de mayor valor: el flujo de un pedido entre recepción y cocina, con seguimiento de su estado en tiempo real, sobre un único local piloto (central, Villa Fátima) y dos roles diferenciados. Quedan fuera de alcance la operación multi-sede, el inventario, el cobro y el cierre de caja, el despacho como rol independiente y el rol administrador; todo ello queda como trabajo futuro, que se desarrolla en el apartado 3.2 del E4.
Enlace al repositorio	https://github.com/jorgeluisgutierrezrivera/max-pizzapp
Enlace al tablero de tareas	https://github.com/users/jorgeluisgutierrezrivera/projects/2
Sistema desplegado	https://maxpizzapp.tech · API: https://maxpizzapp.tech/api/v1/salud · APK de cocina: https://github.com/jorgeluisgutierrezrivera/max-pizzapp/releases/latest/download/max-pizzapp-cocina.apk
Usuarios de prueba	recepcion.demo (rol recepción) y cocina.demo (rol cocina), con datos ficticios; la contraseña se entrega en el campo de texto de la tarea
Pruebas automatizadas	cd backend && npm ci && npm test · reporte en docs/pruebas/reportes/api.txt

Campo	Dato
Fecha de entrega	3 de octubre de 2026 (E3) · E2 entregado el 26 de septiembre · perfil entregado el 19 de septiembre

Datos generales del proyecto.

ÍNDICE DE CONTENIDO

ÍNDICE DE CONTENIDO	iii
ÍNDICE DE TABLAS	vii
ÍNDICE DE FIGURAS.....	viii
ÍNDICE DE ANEXOS.....	ix
RESUMEN.....	x
CAPÍTULO 1. INTRODUCCIÓN	3
1.1 Antecedentes	3
1.2 Descripción del problema	5
1.3 Planteamiento del problema	7
1.4 Justificación del problema	8
1.5 Objetivos	8
1.5.1 Objetivo general.....	8
1.5.2 Objetivos específicos	9
CAPÍTULO 2. MARCO TEÓRICO	11
2.1 Fundamentos conceptuales y tecnológicos	11
2.1.1 Arquitectura cliente-servidor y organización modular	11
2.1.2 Servicios web y comunicación mediante HTTP.....	12
2.1.3 Persistencia relacional, integridad y transacciones	12
2.1.4 Comunicación en tiempo real y modelo orientado a eventos	13
2.1.5 Autenticación, autorización y gestión de identidad	14
2.1.6 Desarrollo multiplataforma y adaptación de interfaz.....	15
2.1.7 Contenerización y separación de servicios	16

2.2 Metodología de desarrollo	16
2.2.1 Uso de asistentes de inteligencia artificial	18
2.2.2 Evidencia de aplicación de la metodología	19
2.3 Análisis de requisitos.....	19
2.3.1 Actores y perfiles de usuario	19
2.3.2 Requisitos funcionales	20
2.3.3 Requisitos no funcionales	21
2.3.4 Reglas de negocio relevantes.....	22
2.3.5 Casos de uso	23
CU-01 Iniciar sesión	23
CU-02 Registrar pedido	24
CU-03 Actualizar el estado del pedido	24
2.4 Diseño de la solución	25
2.4.1 Arquitectura del sistema	25
2.4.2 Modelo de datos	27
Diccionario de datos — cliente	28
Diccionario de datos — pedido	29
Diccionario de datos — detalle_pedido.....	29
Diccionario de datos — producto	30
Diccionario de datos — historial_estado	30
2.4.3 Diseño de la interfaz	30
2.4.4 Diseño de la interfaz de programación de aplicaciones	31
2.5 Justificación del stack tecnológico	32
2.6 Implementación	33

2.6.1 Organización del código	34
2.6.2 Avance por tarjeta.....	35
2.6.3 El flujo implementado: del mostrador a la cocina	36
2.6.4 Decisiones de implementación	37
2.6.5 Interfaz implementada	38
2.6.6 Fragmentos de código ilustrativos.....	40
2.6.7 Dificultades y su resolución.....	41
2.7 Seguridad.....	42
2.7.1 Autenticación y gestión de sesiones	42
2.7.2 Autorización por rol	43
2.7.3 Validación de entradas y vulnerabilidades habituales	44
2.7.4 Cifrado de credenciales y comunicaciones	45
2.7.5 Gestión de variables sensibles	46
2.8 Pruebas.....	46
2.8.1 Plan de pruebas.....	46
2.8.2 Pruebas automatizadas	47
2.8.3 Casos de prueba	48
2.8.4 Requisitos no funcionales	50
2.8.5 Alcance y límites de las pruebas.....	52
2.9 Despliegue	52
CAPÍTULO 3. CONCLUSIONES Y RECOMENDACIONES.....	55
3.1 Conclusiones.....	55
3.2 Recomendaciones.....	55
BIBLIOGRAFÍA CONSULTADA.....	56

ANEXOS	60
Anexo A. Manual de usuario	60
Anexo B. Manual de instalación y despliegue	60
Anexo C. Enlace al repositorio de código	60
Anexo D. Dirección pública del sistema desplegado o archivo instalable .	60
Anexo E. Diagramas en tamaño legible	61
Anexo F. Documentación complementaria pertinente	61

ÍNDICE DE TABLAS

Tabla 1. Soluciones informáticas existentes en el dominio	5
Tabla 2. Cronograma efectivamente ejecutado.....	17
Tabla 3. Actores del sistema.....	20
Tabla 4. Requisitos funcionales, criterios y prioridad	21
Tabla 5. Requisitos no funcionales	22
Tabla 6. Reglas de negocio relevantes	22
Tabla 7. Componentes de la arquitectura	27
Tabla 8. Diccionario de datos de cliente	28
Tabla 9. Diccionario de datos de pedido	29
Tabla 10. Diccionario de datos de detalle_pedido.....	29
Tabla 11. Diccionario de datos de producto	30
Tabla 12. Diccionario de datos de historial_estado	30
Tabla 13. Contrato vigente de la API	32
Tabla 14. Stack tecnológico y criterios de selección	33
Tabla 15. Tarjetas implementadas al corte del E3	35
Tabla 16. Operaciones CRUD sobre el pedido	37
Tabla 17. Validación doble de los datos principales.....	44
Tabla 18. Riesgos del OWASP Top 10:2025 y su tratamiento	45
Tabla 19. Niveles de prueba	47
Tabla 20. Casos de prueba y resultados.....	50
Tabla 21. Mediciones de RNF-01 en producción, 02/10/2026.....	50

ÍNDICE DE FIGURAS

Figura 1. Estado del tablero Kanban al cierre del incremento 2, 25 de septiembre de 2026	18
Figura 2. Diagrama de casos de uso	23
Figura 3. Arquitectura actual de Max Pizzapp	26
Figura 4. Modelo entidad-relación vigente	27
Figura 5. Prototipos de pantallas y adaptación	31
Figura 6. Formulario de venta de Recepción, con una venta lista para confirmar	39
Figura 7. El APK de Cocina en un teléfono Android: la cola en vivo (izquierda), un pedido que llega solo (centro) y el aviso de canal caído con Recargar (derecha).....	40
Figura 8. Las transiciones de estado y el rol que hace cada una	41
Figura 9. El latido del canal y el corte de la conexión al vencer el token (fragmento)	41
Figura 10. La venta, los pedidos de Recepción y la cola de Cocina a 768 × 1024 px, en Edge 154.....	52

ÍNDICE DE ANEXOS

Anexo A. Manual de usuario	60
Anexo B. Manual de instalación y despliegue	60
Anexo C. Enlace al repositorio de código	60
Anexo D. Dirección pública del sistema desplegado o archivo instalable.....	60
Anexo E. Diagramas en tamaño legible	61
Anexo F. Documentación complementaria pertinente	61

RESUMEN

CAPÍTULO 1

CAPÍTULO 1. INTRODUCCIÓN

Max Pizzapp se desarrolla para apoyar la gestión de pedidos de Max's Pizzas en el local central de Villa Fátima, Tarija. El estudio se concentra en el intercambio de información entre dos perfiles operativos: Recepción, que registra y entrega los pedidos, y Cocina, que organiza la preparación y comunica su avance. Las otras dos sucursales del negocio, ubicadas en Villa Avaroa y Tabladita, no forman parte del piloto.

Este capítulo parte del contexto tecnológico de la gestión gastronómica y contrasta soluciones existentes con las necesidades del negocio. Luego describe el proceso manual, formula el problema y justifica el alcance del proyecto. La secuencia concluye con los objetivos que orientan el desarrollo del trabajo, sin atribuir al sistema resultados operativos que todavía no hayan sido medidos.

1.1 Antecedentes

En un establecimiento gastronómico, el pedido enlaza la atención al cliente con la preparación en cocina. Tan y Chang (2010) describen el proceso de servicio como una secuencia en la que el personal registra la orden y la transmite a cocina; en el sistema estudiado, esa información se digitaliza y se envía de forma inmediata al área de preparación. Este antecedente sitúa la gestión del pedido como un problema de continuidad de la información entre quienes toman la orden y quienes la preparan, aunque cada establecimiento deba adaptar el flujo a su propia operación.

Las pantallas de cocina constituyen una forma documentada de apoyar esa continuidad. Oracle (s. f.) define un sistema de pantalla de cocina como un dispositivo que comunica la información del pedido al área de preparación y la muestra en monitores para que el personal conozca qué debe preparar; su documentación también contempla estados, temporizadores y detalles de

cada orden. Estas funciones permiten presentar la comanda y su avance dentro del flujo de cocina, sin suponer que una misma configuración responda a las reglas de todos los negocios.

En este dominio conviven plataformas comerciales de gestión gastronómica, aplicaciones de pantalla de cocina y sistemas de punto de venta de código abierto. Sus alcances son distintos: algunas integran mesas, caja, inventario y reparto, mientras otras se concentran en presentar las órdenes que llegan a cocina. Esa variedad permite comparar qué necesidades generales ya están cubiertas y cuáles requieren una adaptación al contexto del piloto.

La Tabla 1 resume tres soluciones representativas a partir de la documentación pública de sus responsables. La comparación considera la comunicación del pedido hacia cocina, el seguimiento de su preparación y el ajuste al flujo específico de Max's Pizzas.

La comparación no pretende descalificar las plataformas existentes ni afirmar ausencias que sus fuentes no permitan demostrar. Se limita a identificar las capacidades documentadas y a señalar, de forma prudente, los aspectos del proceso de Max's Pizzas que no se describen en la información pública consultada.

Solución	Alcance	Qué resuelve	Qué deja sin resolver	Fuente
Fudo	Plataforma comercial en línea para la gestión gastronómica, con ventas, comandas, pantalla de cocina, inventario, caja, reportes y reparto.	Documenta comandas digitales, tiempos de preparación, alertas de nuevos pedidos y aviso al camarero cuando una orden está lista.	Su alcance comprende procesos que quedan fuera del piloto y se ofrece mediante planes comerciales. La documentación pública consultada no describe las reglas particulares de venta, adición y cancelación de Max's Pizzas.	Fudo (s. f.)

Solución	Alcance	Qué resuelve	Qué deja sin resolver	Fuente
Loyverse KDS	Aplicación de pantalla de cocina para iPad y tabletas Android, conectada al punto de venta Loyverse POS.	Muestra artículos, cantidades, modificadores, notas y tiempo de espera; permite marcar artículos o pedidos completos y emite alertas sonoras.	Depende de Loyverse POS y de una aplicación para tableta. En la documentación pública consultada no se identifica una aplicación web adaptada a los dos roles y reglas del local piloto.	Loyverse (s. f.)
Open Source Point of Sale (OSPOS)	Punto de venta web de código abierto. La página oficial de publicaciones identifica la versión 3.4.1 como la estable más reciente.	Documenta ventas, inventario, clientes, proveedores, registros, reportes y control de permisos en una instalación administrable por quien la despliega.	En la documentación pública consultada no se identifica una pantalla de cocina ni un flujo de estados entre Recepción y Cocina configurado para el proceso del piloto.	Open Source Point of Sale (s. f.)

Tabla 1. Soluciones informáticas existentes en el dominio. Fuente: elaboración propia con base en Fudo (s. f.), Loyverse (s. f.) y Open Source Point of Sale (s. f.).

Fudo y Loyverse KDS documentan el envío de pedidos a pantallas de cocina y el seguimiento de la preparación. Open Source Point of Sale ofrece un punto de venta web con funciones de ventas, inventario, registros y permisos, pero en la documentación pública revisada no se identifica una pantalla de cocina orientada al intercambio de estados entre los dos puestos del piloto.

El vacío que aborda Max Pizzapp no es la inexistencia de software para restaurantes. Surge de la necesidad de representar el proceso concreto del local central de Villa Fátima con dos perfiles operativos, Recepción y Cocina, y con un flujo compartido del pedido. El piloto se limita a ese intercambio y no pretende sustituir las funciones de caja, inventario, reparto, promociones condicionadas ni operación multisede que ofrecen otras plataformas.

1.2 Descripción del problema

Max's Pizzas opera tres locales en la ciudad de Tarija. El estudio toma como piloto el local central de Villa Fátima, cuyo salón dispone de ocho mesas y que además recibe pedidos por mensajería instantánea y para reparto a domicilio. La logística del reparto corresponde a un servicio externo y no forma parte del

sistema. En la entrevista realizada el 18 de septiembre de 2026, la responsable informó un horario de atención de 18:00 a 23:30, que volvió a confirmar el 25 de septiembre de 2026.

En el proceso previo al sistema, Recepción anotaba el pedido en papel y lo pasaba de palabra a Cocina. La preparación avanzaba con la información que el personal alcanzaba a escuchar o recibir, sin una vista compartida que indicara qué pedidos estaban pendientes, en preparación o listos. Por ello, Recepción debía consultar directamente a Cocina para responder al cliente y no contaba con un registro común desde el cual seguir cada cambio.

La entrevista presencial realizada a Herlinda Duran, dueña de Max's Pizzas, el 18 de septiembre de 2026, permitió dimensionar este contexto. Todos los valores siguientes son estimaciones de la responsable sobre la operación de la semana previa y no mediciones instrumentadas del negocio. Según su apreciación, la mayor demanda se concentraba entre las 19:30 y las 22:00, franja en la que se atendían entre 50 y 70 pedidos sumando salón, mensajería y reparto. La duración habitual de un pedido se situaba entre 12 y 15 minutos desde la toma hasta la entrega.

El primer punto crítico era la falta de visibilidad sobre las demoras. Al no existir un estado compartido, Recepción podía advertir que un pedido se había retrasado recién cuando el cliente consultaba o reclamaba. La responsable estimó entre seis y siete pedidos demorados por día durante la semana tomada como referencia.

El segundo aparecía cuando un producto ya no estaba disponible. Esa situación no se reflejaba en un registro compartido, por lo que podía descubrirse recién al iniciar la preparación. En esos casos era necesario contactar al cliente, solicitar una sustitución y reorganizar el trabajo; la responsable estimó que este recorrido añadía entre 10 y 15 minutos al pedido afectado.

El tercer punto crítico provenía del traspaso verbal y del uso de papeles separados. La responsable estimó que entre dos y tres pedidos por día podían prepararse de forma incorrecta, traspapelarse u olvidarse. Además del inconveniente para el cliente, la ausencia de un registro común dificultaba reconstruir qué información se había recibido y en qué momento se había producido el desvío (Entrevista a Herlinda Duran, dueña de Max's Pizzas, 18 de septiembre de 2026).

En conjunto, los datos describen un problema de coordinación y trazabilidad entre los dos puestos que intervienen en el pedido. No constituyen una línea base obtenida automáticamente ni permiten atribuir causas con precisión estadística; se emplean para caracterizar el contexto inicial y deberán contrastarse con mediciones posteriores si se desea evaluar el efecto del sistema.

1.3 Planteamiento del problema

El local central de Max's Pizzas no disponía de un medio compartido que permitiera a Recepción registrar los pedidos y a Cocina comunicar su avance sobre una misma fuente de información. En la semana previa a la entrevista del 18 de septiembre de 2026, la responsable estimó entre seis y siete pedidos demorados y entre dos y tres pedidos mal preparados, traspapelados u olvidados por día. Estas cifras caracterizan el proceso manual y no constituyen resultados medidos por Max Pizzapp. La necesidad central consiste en reducir la dependencia de consultas verbales y mantener una referencia común sobre el estado de cada pedido. A partir de este diagnóstico se formula la siguiente pregunta:

¿Cómo mejorar la gestión y trazabilidad de los pedidos entre Recepción y Cocina del local central de Max's Pizzas, reduciendo la dependencia de la coordinación verbal y permitiendo conocer oportunamente el estado de cada pedido?

1.4 Justificación del problema

La factibilidad técnica del proyecto se sustenta en la disponibilidad de tecnologías web, herramientas de desarrollo e infraestructura para publicar la solución. También se cuenta con acceso al proceso del negocio y con equipos de escritorio o tabletas adecuados para los puestos considerados. Al limitar el piloto al local central de Villa Fátima y a los perfiles de Recepción y Cocina, el desarrollo y la verificación pueden concentrarse en el recorrido principal del pedido dentro de un alcance viable.

La propuesta es pertinente porque responde directamente a la dificultad observada en el intercambio de información entre Recepción y Cocina. El sistema se concentra en el registro y el seguimiento compartido del pedido entre esos dos perfiles, en correspondencia con el problema diagnosticado. Este alcance acotado evita incorporar módulos ajenos al flujo central del piloto.

El impacto esperado consiste en disponer de una referencia compartida del pedido, conservar la trazabilidad de sus cambios y diferenciar las acciones que corresponden a Recepción y Cocina. Estas capacidades pueden verificarse técnicamente y podrían disminuir la necesidad de consultas verbales para conocer el avance del pedido y responder al cliente. Sin embargo, todavía no existe evidencia suficiente para afirmar una reducción comprobada de demoras, errores o tiempos de atención; ese efecto deberá evaluarse mediante datos obtenidos durante el uso operativo y comparados con el proceso previo.

1.5 Objetivos

1.5.1 Objetivo general

Desarrollar un sistema web y móvil, con interfaz adaptable, para gestionar los pedidos entre los perfiles de Recepción y Cocina del local central de Max's

Pizzas, con trazabilidad de estados y sincronización de la información en tiempo real.

1.5.2 Objetivos específicos

1. Especificar los requisitos del proceso de toma y preparación de pedidos de Max's Pizzas mediante 14 requisitos funcionales con criterios de aceptación y 5 requisitos no funcionales con métricas verificables, priorizados con MoSCoW.
2. Definir la arquitectura de la solución y el modelo de datos relacional de cinco entidades de Max Pizzapp, con su diccionario de datos y el contrato de la API.
3. Implementar los módulos de Recepción y Cocina con sincronización en tiempo real del estado y los cambios del pedido.
4. Aplicar mecanismos de autenticación, autorización por rol y protección de las comunicaciones de Max Pizzapp.
5. Ejecutar pruebas funcionales, de integración y de rendimiento que permitan verificar el flujo de pedidos y la sincronización en tiempo real.
6. Desplegar Max Pizzapp en una dirección pública accesible y documentar el procedimiento de despliegue.

CAPÍTULO 2

CAPÍTULO 2. MARCO TEÓRICO

El presente capítulo desarrolla los fundamentos y decisiones técnicas que sustentan Max Pizzapp. Se parte de los conceptos necesarios para comprender su arquitectura y tecnologías y, posteriormente, se describe la metodología aplicada, los requisitos, el diseño, la selección del stack, la implementación, los mecanismos de seguridad, las pruebas y el despliegue de la solución.

2.1 Fundamentos conceptuales y tecnológicos

2.1.1 Arquitectura cliente-servidor y organización modular

La arquitectura de software describe los elementos principales de un sistema, sus relaciones y las decisiones que condicionan su evolución (Bass, Clements y Kazman, 2021). En una aplicación cliente-servidor, el cliente presenta información, recoge datos y envía solicitudes; el servidor interpreta esas solicitudes, autoriza la operación, aplica las reglas de negocio y coordina la persistencia. Esta frontera separa presentación, aplicación y datos, y evita que cada interfaz aplique reglas distintas.

La base de datos permanece detrás del servidor y no se expone directamente a los clientes. El servidor de aplicación organiza sus responsabilidades en módulos internos y se despliega como una unidad. Esta modularidad separa el código por funciones sin convertir cada módulo en un proceso independiente; en Max Pizzapp, las reglas del negocio permanecen concentradas en el servidor antes de acceder a la persistencia.

En Max Pizzapp, los clientes web y Android se comunican con el servidor de aplicación, que concentra las reglas antes de acceder a la persistencia. Este servidor se ejecuta como una unidad sobre Node.js 24.15.0 LTS y Express 5.2.1, con responsabilidades internas separadas. La descripción aclara por

qué la interfaz no consulta directamente las tablas; la valoración de la arquitectura frente a alternativas corresponde a los apartados de diseño y justificación tecnológica.

2.1.2 Servicios web y comunicación mediante HTTP

Una interfaz de programación de aplicaciones (*Application Programming Interface*, API) establece cómo un cliente solicita capacidades a otro componente. HTTP organiza la interacción mediante solicitudes y respuestas: el cliente identifica un recurso y expresa una intención; el servidor devuelve un código de estado y, cuando corresponde, una representación. Como protocolo sin estado, cada solicitud debe comprenderse por su contenido y por el contexto transmitido expresamente (Fielding, Nottingham y Reschke, 2022).

Los métodos aportan significado: GET obtiene una representación sin pedir una modificación; POST solicita que el recurso procese el contenido enviado; PATCH aplica cambios parciales a un recurso (Dusseault y Snell, 2010). Los códigos de estado distinguen éxito, redirección y errores. El contrato de la API completa ese vocabulario con recursos, parámetros, cuerpos, respuestas y errores esperados, sin revelar cómo se ejecuta cada regla.

OpenAPI 3.0.3 proporciona una descripción independiente del lenguaje para que personas y herramientas comprendan una API HTTP sin inspeccionar su código (OpenAPI Initiative, 2020). Max Pizzapp conserva su contrato en esa versión: los clientes de Recepción y Cocina envían solicitudes HTTP y Express las recibe en el servidor Node.js. Las rutas concretas se presentan en el diseño; aquí interesa la frontera que permite a cliente y servidor compartir un contrato sin compartir su implementación.

2.1.3 Persistencia relacional, integridad y transacciones

El modelo relacional representa la información mediante relaciones que, en una base de datos práctica, se expresan como tablas de filas y columnas.

Codd (1970) lo formuló para separar la representación lógica de los datos de su organización física. Una clave primaria identifica cada fila; una clave foránea relaciona tablas mediante valores, sin exigir que la aplicación conozca dónde están almacenados los registros.

La integridad se apoya en restricciones de obligatoriedad, unicidad y validación. Las claves foráneas impiden referencias hacia filas inexistentes y sostienen la integridad referencial. PostgreSQL 17.11 aplica estos controles al almacenar los datos, por lo que la base participa en la conservación de reglas estructurales y no funciona solo como depósito (PostgreSQL Global Development Group, s. f.b).

Una transacción agrupa varias operaciones como una unidad: la confirmación (COMMIT) hace permanentes los cambios y la reversión (ROLLBACK) los descarta. Esta atomicidad evita guardar, por ejemplo, una cabecera sin todas sus líneas; el aislamiento controla qué estados observan sesiones concurrentes (PostgreSQL Global Development Group, s. f.a). Max Pizzapp utiliza PostgreSQL 17.11 para conservar la cabecera, las líneas, el total y el historial del pedido. Las entidades y restricciones concretas se presentan en el diseño.

2.1.4 Comunicación en tiempo real y modelo orientado a eventos

Una API basada solo en solicitudes del cliente no informa por sí misma que otro usuario cambió un dato. El sondeo periódico (*polling*) consulta cada cierto intervalo, con un compromiso entre actualidad y tráfico: un intervalo largo retrasa la vista y uno corto genera solicitudes aunque nada cambie. Un canal persistente permite que el servidor notifique el cambio cuando ocurre.

WebSocket habilita la comunicación bidireccional mediante una conexión entre cliente y servidor; tras el intercambio inicial, ambos extremos envían mensajes sin abrir una solicitud HTTP por cada notificación (Fette y Melnikov,

2011). El modelo orientado a eventos (*event-driven*) representa los cambios mediante mensajes identificables. Socket.IO 4.8.3 implementa este intercambio y puede usar WebSocket como transporte; no es sinónimo del protocolo, pues añade su propio formato, otros transportes y reconexión automática (Socket.IO, 2026). La aplicación emplea `socket_io_client` 3.1.6.

Una reconexión no demuestra que todos los eventos fueron recibidos. El evento avisa que algo ocurrió, mientras la base conserva la fuente persistente de verdad desde la cual debe reconstruirse la vista. En Max Pizzapp, Recepción y Cocina reciben avisos de pedidos y cambios de estado sin recargar manualmente, y pueden recuperar el estado almacenado tras una interrupción. La medición del tiempo de propagación se reserva para las pruebas.

2.1.5 Autenticación, autorización y gestión de identidad

La autenticación establece quién es la persona usuaria y la autorización determina qué puede hacer. Un proveedor de identidad administra las credenciales y autentica, mientras la aplicación aplica las decisiones de acceso sobre sus recursos. OpenID Connect Core 1.0 añade una capa de identidad sobre OAuth 2.0 y distingue el token de identidad (ID Token), que comunica afirmaciones sobre la identidad autenticada, del token de acceso (Access Token), que autoriza solicitudes a recursos protegidos (Sakimura et al., 2023).

En el flujo de código de autorización (*Authorization Code*), el navegador vuelve a la aplicación con un código de uso limitado que luego se canjea. La prueba de clave para intercambio de código (*Proof Key for Code Exchange*, PKCE) añade un verificador del cliente y un desafío derivado de él. Quien intercepte solo el código no puede canjearlo sin el verificador; PKCE mitiga ese riesgo en clientes públicos, pero no cifra el token ni reemplaza HTTPS (Sakimura, Bradley y Agarwal, 2015).

Max Pizzapp utiliza Keycloak 26.7.4 como proveedor de identidad con OpenID Connect, código de autorización y PKCE; Keycloak documenta la redirección al proveedor y el canje del código (Keycloak, s. f.). Los roles Recepción y Cocina expresan permisos distintos, pero el servidor debe comprobar el rol antes de ejecutar una acción. Las validaciones de los tokens y los controles por operación se desarrollan en seguridad.

2.1.6 Desarrollo multiplataforma y adaptación de interfaz

Flutter admite despliegues para Android y web, entre otras plataformas (Flutter, 2026b). Esto permite compartir una base de código, aunque cada destino conserva capacidades y restricciones propias. La interfaz también debe responder al espacio disponible y al modo de interacción: el diseño responsivo (responsive) reorganiza elementos según la ventana, mientras el diseño adaptable (adaptive) selecciona una disposición adecuada al contexto de uso (Flutter, 2026a).

Compartir una base de código no obliga a mostrar la misma composición en todos los destinos. Los datos, las reglas de interacción y los componentes reutilizables pueden mantenerse, mientras los puntos ligados al navegador o al sistema operativo se resuelven mediante adaptaciones delimitadas. Esta organización permite conservar un proyecto común sin desconocer las diferencias de cada plataforma.

En Max Pizzapp, Recepción utiliza la versión web principalmente desde computadora. Durante la tutoría T3 se incorporó al alcance una compilación Android para Cocina, destinada a celular o tableta y derivada de la misma base de código en Flutter 3.44.8 y Dart 3.12.2. La compilación Android se distribuye como un instalable firmado (APK) y se verificó en un teléfono con Android 13 contra el sistema desplegado. Las particularidades de autenticación, sonido y pantalla encendida se describen en la implementación (2.6); la firma y la distribución, en el despliegue (2.9).

2.1.7 Contenerización y separación de servicios

Un contenedor es un proceso aislado que ejecuta una aplicación con los archivos y dependencias definidos para ella. La imagen es la plantilla desde la que se crea ese proceso; imagen y contenedor no son equivalentes. Docker describe el contenedor como una unidad aislada que comparte el núcleo del sistema anfitrión, a diferencia de una máquina virtual con sistema operativo completo (Docker, s. f.b).

Docker Compose declara servicios, imágenes, redes, volúmenes y configuración en un archivo. Un servicio representa un componente con una imagen y configuración determinadas; la red interna comunica servicios y el volumen conserva datos tras recrear un contenedor (Docker, s. f.a). Esta separación expresa límites de ejecución, no convierte por sí sola a la aplicación en microservicios. Un proxy inverso (reverse proxy) puede recibir solicitudes públicas y dirigir las al servicio correspondiente.

Max Pizzapp ejecuta sus componentes con Docker Engine 29.7.2 en desarrollo y 29.8.1 en el servidor. Docker Compose v5.3.1 en desarrollo y v5.5.1 en el servidor coordina la aplicación, la base y el proveedor de identidad. Los nombres de contenedores, redes, volúmenes, puertos y el procedimiento de publicación se reservan para despliegue.

2.2 Metodología de desarrollo

Max Pizzapp se desarrolló mediante Kanban, con un tablero en GitHub Projects organizado en las columnas Por hacer, En curso, En pruebas y Hecho. El trabajo fue individual y se aplicó un límite de trabajo en curso de una tarjeta (WIP = 1). Esta elección hizo visible el estado de cada tarea y permitió terminar su implementación y sus pruebas antes de iniciar la siguiente. No se adoptó Scrum porque el proyecto no ejecutó roles ni ceremonias de equipo propios de ese marco.

El flujo fue continuo y produjo entregas incrementales. Cada tarjeta siguió la secuencia: plan escrito, revisión y aprobación del plan, implementación, pruebas, revisión del resultado y cierre con evidencia. Los planes se conservaron en docs/planes/. Junto con el tablero, la bitácora de decisiones, el registro de errores y las pruebas, estos documentos permiten reconstruir la evolución del proyecto.

La Tabla 2 resume el cronograma por semanas del calendario académico. Cada semana agrupa los incrementos y las tarjetas que se cerraron en ella; dentro de cada semana, el trabajo avanzó en flujo continuo, una tarjeta a la vez.

Semana	Periodo	Contenido ejecutado o previsto	Observación	Estado
Semana 1	11–20 sep	Incremento 0: alcance, requisitos, arquitectura, modelo inicial, repositorio y perfil del E1. RF-01 a RF-14 y RNF-01 a RNF-05, como especificación.	Inicio y delimitación del proyecto.	Completada
Semana 2	21–27 sep	Incrementos 1 y 2 (tarjetas 01 a 06): persistencia, despliegue HTTPS, identidad, API, carta y flujo Recepción–Cocina–Recepción. RF-01 a RF-09, RF-11 y RF-14.	Lo previsto para la Semana 3 sobre pedidos se adelantó y cerró en la tarjeta 06.	Completada
Semana 3	28 sep–3 oct	Incrementos 3 y 4: aviso de canal caído y sesión (07), APK de Cocina (12), endurecimiento de seguridad (09) y pruebas con evidencia (10). RF-03, RF-04, RF-06 a RF-08 y RNF-01 a RNF-05.	RF-13 (tarjeta 08) pasa a la Semana 4.	Completada
Semana 4	5–10 oct	Previsto: RF-13, documentación del despliegue, manuales, conclusiones y cierre del documento.	Posterior al corte del E3.	Por iniciar

Tabla 2. Cronograma efectivamente ejecutado al 3 de octubre de 2026. Fuente: elaboración propia.

La Figura 1 conserva la evidencia disponible del tablero al cierre del incremento 2, el 25 de septiembre de 2026. En ese corte histórico se registraban 25 tarjetas: 18 en Hecho, 7 en Por hacer y ninguna en En curso o En pruebas. La captura no representa el estado al corte del E3; se utiliza para demostrar la aplicación del flujo y del tablero en una fecha verificable.

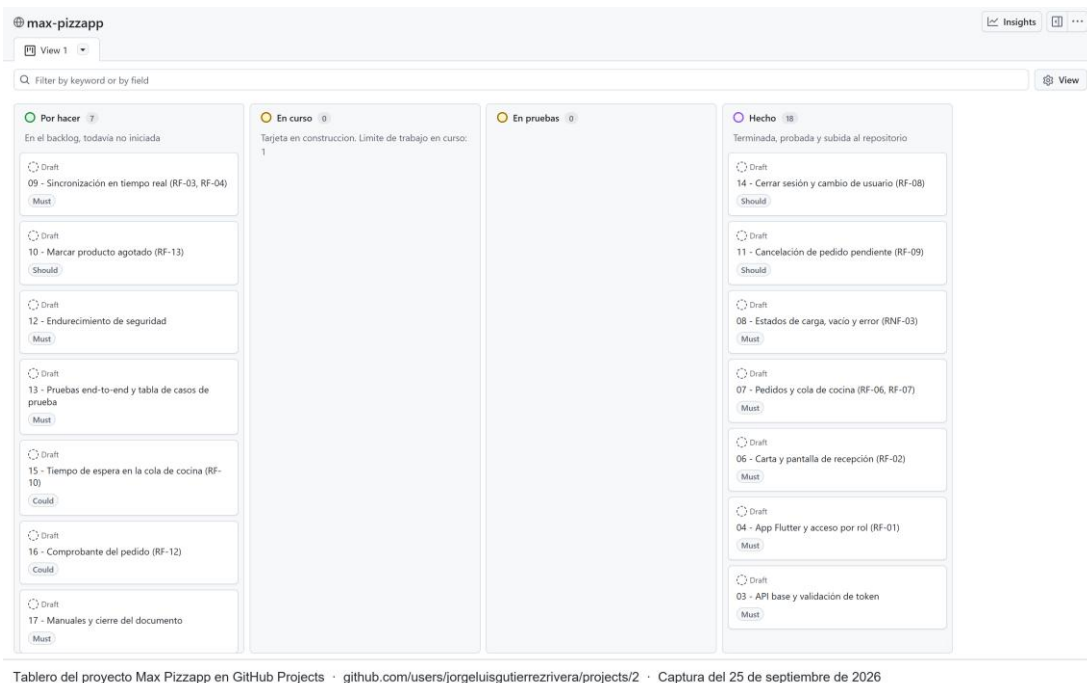


Figura 1. Estado del tablero Kanban al cierre del incremento 2, 25 de septiembre de 2026.

Fuente: captura propia de GitHub Projects.

2.2.1 Uso de asistentes de inteligencia artificial

Durante el desarrollo se utilizaron asistentes de inteligencia artificial como herramientas de apoyo. Claude Code, de Anthropic, se empleó para proponer borradores de código y documentación técnica, apoyar la revisión y depuración de implementaciones, explorar alternativas, revisar configuraciones y generar casos de prueba. ChatGPT, de OpenAI, se utilizó para revisar la coherencia documental, contrastar el contenido con las plantillas institucionales, organizar apartados y mejorar la redacción.

Las decisiones finales de alcance, arquitectura y diseño, así como la selección, adaptación, integración y validación del código incorporado al proyecto, fueron responsabilidad del autor. Todo resultado generado con apoyo de estas herramientas fue revisado, ejecutado y verificado antes de su incorporación. El autor mantiene la responsabilidad sobre el funcionamiento del sistema y el

contenido del documento, y no incorporó elementos que no pudiera explicar y defender técnicamente.

2.2.2 Evidencia de aplicación de la metodología

La aplicación de Kanban se sustenta en el tablero con sus columnas y tarjetas, el cronograma ejecutado de la Tabla 2, los planes previos conservados en docs/planes/, la bitácora de decisiones, el registro de errores y soluciones, y las suites y evidencias de pruebas asociadas a los incrementos. En conjunto, estos elementos permiten reconstruir qué se planificó, qué se ejecutó y con qué evidencia se cerró cada tarjeta hasta el corte del 3 de octubre de 2026.

2.3 Análisis de requisitos

2.3.1 Actores y perfiles de usuario

El alcance reconoce exactamente dos actores. La Tabla 3 describe capacidades especificadas por los requisitos; por ello, la inclusión de RF-13 no implica que su interfaz ya esté implementada.

ID	Actor	Descripción	Qué puede hacer	Qué no puede hacer
A-01	Recepción	Personal que toma y gestiona pedidos desde el mostrador.	Iniciar y cerrar sesión; consultar la carta; registrar y seguir pedidos; recibir aviso de listo; marcar entrega; cancelar solo en pendiente; registrar observaciones; agregar productos según el estado; marcar un producto como agotado según RF-13.	Iniciar preparación o marcar listo; borrar pedidos físicamente; administrar usuarios, inventario, caja o sucursales.
A-02	Cocina	Personal que recibe y prepara pedidos en orden de llegada.	Iniciar y cerrar sesión; ver cola y observaciones; iniciar preparación; marcar listo; recibir actualizaciones en vivo; usar el cliente Android; marcar un producto como agotado según RF-13.	Registrar ventas; entregar o cancelar pedidos; modificar precios; administrar usuarios o caja; borrar pedidos.

Tabla 3. Actores del sistema. Fuente: elaboración propia con base en los requisitos vigentes.

2.3.2 Requisitos funcionales

Los requisitos funcionales se expresan como historias de usuario y se acompañan de un camino esperado y uno de error o excepción. La priorización MoSCoW conserva 7 requisitos Must, 5 Should y 2 Could; por tanto, los Must representan 50 % del total.

ID	Historia de usuario	Criterios de aceptación	MoSCoW
RF-01	Como usuario, quiero iniciar sesión con mis credenciales para acceder a las funciones de mi rol.	CA-01.1 Con credenciales válidas, el proveedor emite el resultado de autenticación; la aplicación valida el token y dirige al rol. CA-01.2 Con credenciales inválidas, el proveedor rechaza el acceso sin emitir token y no se muestra ninguna pantalla de rol.	Must
RF-02	Como Recepción, quiero registrar un pedido con productos de la carta para enviarlo a Cocina.	CA-02.1 Al confirmar una selección válida, se crea el pedido pendiente y llega a Cocina. CA-02.2 Sin productos, no se crea y se solicita agregar al menos uno.	Must
RF-03	Como Recepción, quiero ver el estado actualizado en tiempo real para conocer el avance sin consultar a Cocina.	CA-03.1 Un cambio de Cocina se refleja en menos de 2 s sin recargar. CA-03.2 Si cae el canal, se muestra un aviso y se permite recargar el estado vigente.	Must
RF-04	Como Recepción, quiero recibir un aviso cuando un pedido quede listo para proceder con la entrega.	CA-04.1 Al marcarse listo, aparece un aviso visible y el pedido se destaca. CA-04.2 Sin pedidos listos, la vista muestra el estado vacío sin avisos pendientes.	Must
RF-05	Como Recepción, quiero marcar un pedido listo como entregado para cerrar su atención.	CA-05.1 Un pedido listo pasa a entregado y sale de la lista activa. CA-05.2 Si aún no está listo, la operación se rechaza e informa el estado.	Must
RF-06	Como Cocina, quiero ver los pedidos entrantes en tiempo real y por orden de llegada para prepararlos en secuencia.	CA-06.1 Un pedido confirmado aparece en menos de 2 s al final de la cola. CA-06.2 Sin pedidos pendientes, se muestra el estado vacío.	Must
RF-07	Como Cocina, quiero pasar un pedido a en preparación y luego a listo para informar el avance.	CA-07.1 Cada transición válida se guarda y propaga a Recepción. CA-07.2 Si el pedido está entregado o cancelado, el cambio se rechaza por ser un estado final.	Must

ID	Historia de usuario	Criterios de aceptación	MoSCoW
RF-08	Como usuario, quiero cerrar sesión y permitir otra sesión en el mismo dispositivo para atribuir cada acción.	CA-08.1 Al cerrar, se descarta el token y se vuelve al acceso. CA-08.2 Con token expirado, la operación se rechaza y se solicita autenticar de nuevo.	Should
RF-09	Como Recepción, quiero cancelar un pedido mientras siga pendiente para corregir la toma sin afectar preparación iniciada.	CA-09.1 En pendiente y con motivo, queda cancelado y se notifica a Cocina. CA-09.2 En preparación, listo o entregado, la cancelación se impide.	Should
RF-10	Como Cocina, quiero ver el tiempo de espera para priorizar los pedidos más antiguos.	CA-10.1 La tarjeta actualiza los minutos desde el registro. CA-10.2 Al superar 15 minutos, el pedido activo muestra además la etiqueta «demorado».	Could
RF-11	Como Recepción, quiero agregar una observación para comunicar indicaciones a Cocina.	CA-11.1 La nota registrada se muestra junto al pedido en Cocina. CA-11.2 Si excede el máximo permitido, se rechaza e informa el límite.	Should
RF-12	Como Recepción, quiero visualizar un comprobante para mostrar al cliente el contenido del pedido.	CA-12.1 Presenta productos, cantidades, total y número. CA-12.2 Para un pedido inexistente, informa «Pedido no encontrado».	Could
RF-13	Como Cocina o Recepción, quiero marcar un producto agotado para impedir nuevas selecciones.	CA-13.1 El producto deja de ser seleccionable y el cambio llega a las pantallas en menos de 2 s. CA-13.2 Los pedidos activos que ya lo contienen no se alteran. Es requisito vigente; no se declara aquí como implementado.	Should
RF-14	Como Recepción, quiero agregar productos a un pedido enviado para no crear otro pedido por una compra adicional.	CA-14.1 En pendiente o en preparación admite pizzas y bebidas; Cocina recibe la actualización. CA-14.2 En listo admite solo bebidas; entregado o cancelado no admite agregados. Las líneas enviadas no se modifican ni eliminan: solo se añaden nuevas.	Should

Tabla 4. Requisitos funcionales, criterios de aceptación y prioridad. Fuente: elaboración propia con base en el backlog vigente.

2.3.3 Requisitos no funcionales

Los cinco requisitos no funcionales establecen condiciones medibles para el comportamiento del sistema. En este apartado se registran los criterios; su ejecución y sus resultados se documentarán en los apartados de seguridad y pruebas.

ID	Tipo	Enunciado	Criterio de verificación
RNF-01	Rendimiento	Los cambios deben propagarse sin intervención y el listado principal responder con fluidez bajo la carga definida.	Cambio de estado < 2 s en 30 repeticiones, reportando el peor caso; GET /api/v1/pedidos < 2 s con 50 pedidos activos y 5 usuarios concurrentes, medido con k6 y percentil 95. Son criterios, no resultados.
RNF-02	Seguridad	La identidad se gestiona con OpenID Connect y Keycloak; las comunicaciones públicas van por HTTPS y el servidor autoriza por rol.	Token válido en operaciones protegidas; validación de firma y expiración; verificación de rol y transición; transporte cifrado. La evidencia de resultados corresponde a 2.7 y 2.8.

ID	Tipo	Enunciado	Criterio de verificación
RNF-03	Usabilidad	La tarea principal debe completarse en tres pasos o menos y toda vista de datos debe expresar su situación.	Registro en ≤ 3 pasos; estados cargando, con datos, vacío y error en las vistas que consumen datos.
RNF-04	Compatibilidad	Recepción opera como cliente web en Chrome y Edge — últimas dos versiones— con diseño adaptable; Cocina se ejecuta como aplicación Android desde la misma base Flutter.	Web sin desplazamiento horizontal a 1366 y 768 px, en Chrome y Edge; en Android, la versión 7 es el mínimo configurado y el APK de Cocina se validó en un Honor 50 con Android 13. No se infiere compatibilidad probada con otras versiones o modelos.
RNF-05	Disponibilidad	El servicio debe estar disponible de 18:00 a 23:30 y degradarse de forma controlada si se pierde el canal en vivo.	GET /api/v1/salud cada 5 min durante ≥ 7 días, con ≥ 99 % de respuestas 200 en la franja; aviso de pérdida del canal < 10 s y recuperación o recarga controlada. Son criterios, no resultados.

Tabla 5. Requisitos no funcionales y criterios de verificación. Fuente: elaboración propia.

2.3.4 Reglas de negocio relevantes

Las reglas de la Tabla 6 condicionan la forma en que se registran y transforman los pedidos; no constituyen módulos adicionales ni sustituyen los requisitos funcionales.

Regla	Aplicación en Max Pizzapp
Pizza entera	Solo se venden pizzas enteras: de un sabor o compuestas por dos mitades de sabores distintos.
Dos sabores	El precio es $(\text{precio A} + \text{precio B}) / 2$. El servidor efectúa el cálculo final y contrasta el total esperado.
Solo sabor único	Dos estaciones, Tres estaciones y Criolla española se venden enteras y no pueden utilizarse como mitad.
Cancelación	Es una baja lógica y solo procede mientras el pedido está pendiente. No existe DELETE como operación normal del pedido.
Agregados	Pendiente o en preparación: pizzas y bebidas; listo: solo bebidas; entregado o cancelado: ninguno. Las líneas ya enviadas permanecen sin modificación ni eliminación.
Venta directa	Una venta compuesta solo por bebidas nace entregada y no pasa por Cocina.
Datos de carta	Los precios de las 15 pizzas están confirmados; bebidas y extras continúan como datos ficticios hasta contar con evidencia posterior.

Tabla 6. Reglas de negocio relevantes. Fuente: elaboración propia con base en las decisiones y el esquema vigentes.

Permanecen fuera del alcance la operación multi-sede, el inventario, la caja y el arqueo, los reportes de ventas, las promociones condicionales y 2×1, el rol Administrador, un despacho independiente, la administración completa de la

carta y un historial administrativo. RF-13 se limita a la disponibilidad simple del producto y no constituye control de inventario.

2.3.5 Casos de uso

Los tres casos oficiales concentran la interacción esencial entre los dos perfiles. La Figura 2 muestra sus asociaciones y sitúa la cancelación como alternativa del cambio de estado, no como parte del camino principal.

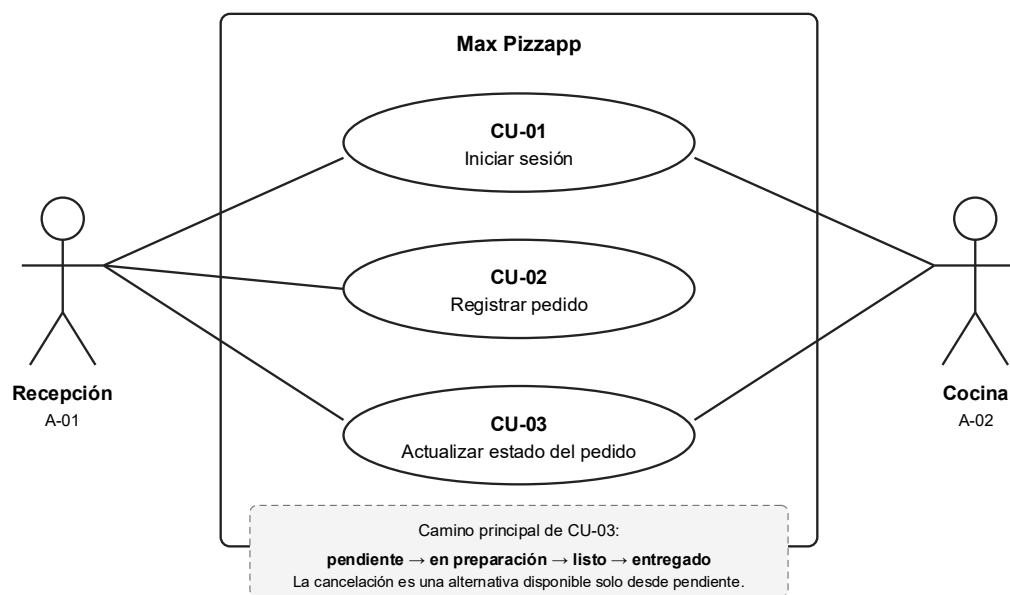


Figura 2. Diagrama de casos de uso de Max Pizzapp. Fuente: elaboración propia con base en los requisitos vigentes.

CU-01 Iniciar sesión

Actor principal: Recepción o Cocina. Precondición: la cuenta está activa en Keycloak y el sistema es accesible. Flujo principal: 1) el usuario abre el cliente; 2) la aplicación inicia el flujo de autorización; 3) Keycloak solicita las credenciales en su propia interfaz; 4) devuelve un código de un solo uso; 5) el cliente lo canjea mediante PKCE; 6) la aplicación y la API validan el token y presentan la vista del rol. Flujos alternativos: A1) credenciales inválidas:

Keycloak rechaza el acceso y no emite token; A2) token expirado: la operación protegida se rechaza y se requiere renovar o iniciar sesión. Postcondición: existe una sesión autenticada con rol Recepción o Cocina; la contraseña no pasa por Max Pizzapp.

CU-02 Registrar pedido

Actor principal: Recepción. Precondición: Recepción inició sesión y dispone de la carta. Flujo principal: 1) registra cliente y selecciona consumo en local o para llevar; 2) agrega una o más pizzas enteras, de un sabor o mitad y mitad, con sus extras, y puede sumar bebidas; 3) añade una observación para Cocina si corresponde; 4) revisa el total y confirma; 5) el servidor valida productos, reglas y precios, guarda el pedido pendiente y emite el evento a Cocina. Flujos alternativos: A1) sin pizza o con datos inválidos, no se crea el pedido y se informa la corrección; A2) si cambió el precio o la disponibilidad, el servidor rechaza la confirmación e indica el conflicto. Postcondición: el pedido queda persistido con sus líneas, total, autor y número del día, visible para ambos perfiles.

CU-03 Actualizar el estado del pedido

Actor principal: Cocina; actor secundario: Recepción. Precondición: existe un pedido activo y cada perfil inició sesión. Flujo principal: 1) Cocina recibe la cola ordenada por llegada; 2) pasa un pedido pendiente a en preparación; 3) al concluir lo marca listo; 4) Recepción recibe el aviso y lo marca entregado; 5) cada cambio se valida, registra y propaga. Flujos alternativos: A1) desde pendiente, Recepción puede cancelar con motivo; A2) una transición no permitida se rechaza sin alterar el estado; A3) si el canal está degradado, el cambio persistido se recupera mediante recarga o reconexión. Postcondición: el pedido queda en el nuevo estado y el historial conserva actor, nombre y fecha; cancelado y entregado son estados finales.

2.4 Diseño de la solución

2.4.1 Arquitectura del sistema

La arquitectura separa los clientes, la aplicación del dominio, la persistencia y la identidad. Recepción utiliza principalmente el cliente web y Cocina cuenta con un destino Android; ambos proceden de la misma base Flutter. La versión web conserva técnicamente las vistas compartidas, pero el destino operativo incorporado para Cocina es Android.

Caddy constituye la entrada pública cifrada y encamina las solicitudes hacia la API, el canal Socket.IO o Keycloak. Node.js y Express aplican las reglas de RF-02, RF-05 y RF-07 antes de persistir; Socket.IO atiende RF-03, RF-04 y RF-06; Keycloak y la validación por JWKS sostienen RNF-02. PostgreSQL conserva el estado recuperable requerido por RNF-05. Ningún cliente se conecta directamente a la base.

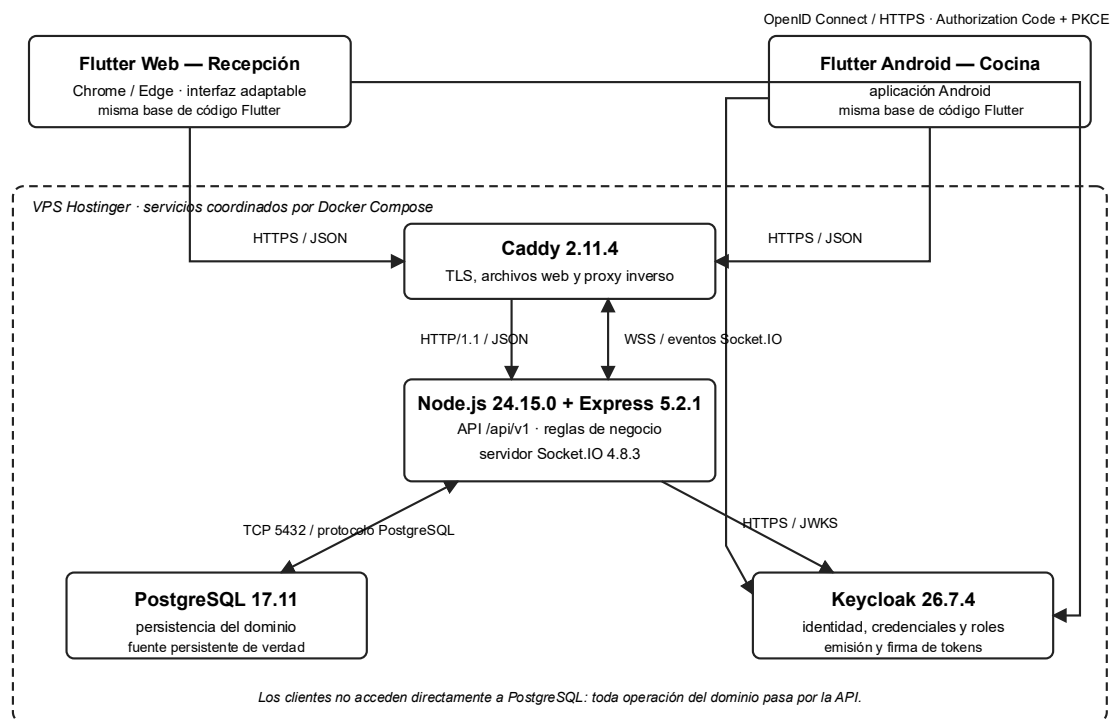


Figura 3. Arquitectura actual de Max Pizzapp. Fuente: elaboración propia con base en la configuración vigente del proyecto.

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Cliente web	Toma y seguimiento de pedidos de Recepción; conserva técnicamente las vistas compartidas.	Flutter 3.44.8 / Dart 3.12.2	Caddy, API, Socket.IO y Keycloak	HTTPS/JSON; WSS/eventos; OIDC/HTTPS
Cliente Android	Cola y actualización de pedidos de Cocina desde el segundo destino Flutter.	Flutter 3.44.8, flutter_appauth 12.1.0	Caddy, API, Socket.IO y Keycloak	HTTPS/JSON; WSS/eventos; OIDC/PKCE
Proxy inverso	Termina TLS, sirve la web y enruta API, Socket.IO e identidad.	Caddy 2.11.4	Clientes, API y Keycloak	HTTPS público; HTTP/WS internos
Servidor de aplicación	Aplica roles, transiciones, precios y persistencia; expone el contrato HTTP.	Node.js 24.15.0 + Express 5.2.1	Caddy, PostgreSQL, Keycloak	HTTP/JSON; TCP PostgreSQL; HTTPS/JWKS
Canal de eventos	Notifica pedidos, agregados, estados y recuperación del canal; forma parte del proceso Node.	Socket.IO 4.8.3	Clientes web y Android	WSS / eventos Socket.IO
Base de datos	Conserva pedidos, líneas, carta e historial como fuente persistente de verdad.	PostgreSQL 17.11	Servidor de aplicación y Keycloak en bases separadas	TCP 5432 / PostgreSQL

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Proveedor de identidad	Autentica, administra credenciales y roles, y firma tokens.	Keycloak 26.7.4	Clientes, API y PostgreSQL	OpenID Connect/HTTPS; JWKS/JSON; SQL interno

Tabla 7. Componentes de la arquitectura y sus responsabilidades. Fuente: elaboración propia.

2.4.2 Modelo de datos

El modelo relacional mantiene cinco entidades del dominio. Keycloak administra usuarios fuera de este modelo; para la trazabilidad, pedido, detalle_pedido e historial_estado conservan el sub y el nombre registrado cuando ocurre la acción. Las relaciones adicionales permiten representar una segunda mitad y vincular cada extra con la línea de su pizza.

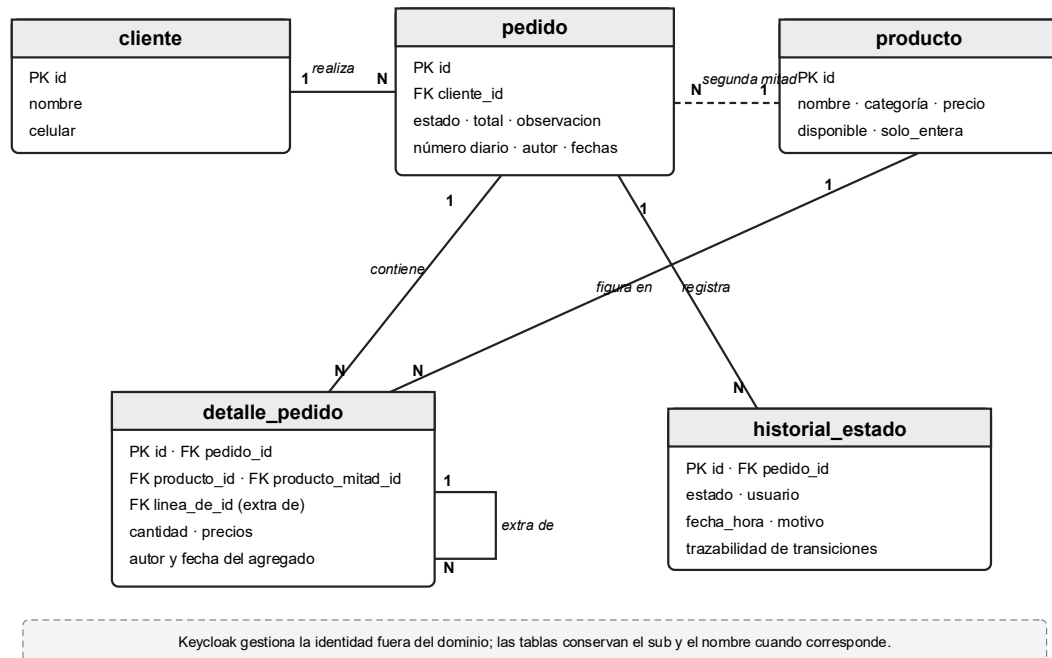


Figura 4. Modelo entidad-relación vigente. Fuente: elaboración propia a partir del esquema y sus migraciones.

PostgreSQL es la fuente persistente de verdad. La cancelación cambia el estado a cancelado y mantiene el pedido y su historial; no existe DELETE como operación normal del negocio. Las reglas ON DELETE CASCADE o RESTRICT expresan integridad referencial del esquema y no habilitan una opción de borrado en la interfaz ni en el contrato de la API.

Diccionario de datos — cliente

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador interno.
nombre	varchar	120	Sí	No vacío	Nombre comunicado por Recepción.
celular	varchar	20	No	Único si existe; ^[5-7][0-9]{7}\$	Número boliviano opcional: 8 dígitos que empiezan con 5, 6 o 7.

Tabla 8. Diccionario de datos de la entidad cliente. Fuente: elaboración propia a partir del esquema vigente.

Diccionario de datos — pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador interno.
cliente_id	integer	—	Condicional	FK cliente; RESTRICT	Nulo solo en venta directa.
numero_del_dia	integer	—	Condicional	Único con día; > 0	Número mostrado; nulo en venta directa.
dia	date	—	Sí	Generado desde creado_en	Día de operación en hora boliviana.
para_llevar	boolean	—	Condicional	Consistencia pedido/venta directa	Modalidad del pedido.
creado_por	text	—	Sí	No vacío	sub de Keycloak que crea.
creado_por_nombre	varchar	120	Sí	—	Nombre copiado al crear.
estado	estado_pedido	—	Sí	Enum; pendiente por defecto	Estado lógico.
observacion	varchar	240	No	—	Indicación para Cocina.
total	numeric	10,2	Sí	>= 0	Total persistido.
creado_en	timestampz	—	Sí	now()	Fecha y hora de creación.

Tabla 9. Diccionario de datos de la entidad pedido. Fuente: elaboración propia a partir del esquema vigente.

Diccionario de datos — detalle_pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK; único con pedido_id	Identificador de línea.
pedido_id	integer	—	Sí	FK pedido; CASCADE	Pedido propietario.
producto_id	integer	—	Sí	FK producto; RESTRICT	Producto principal o extra.
producto_mitad_id	integer	—	No	FK producto; distinto del principal	Segundo sabor de una pizza.
linea_de_id	integer	—	No	FK compuesta al detalle del mismo pedido	Línea de pizza a la que pertenece un extra.
cantidad	integer	—	Sí	1 a 999	Unidades.
precio_unitario	numeric	10,2	Sí	>= 0	Precio guardado al vender.
subtotal	numeric	10,2	Sí	>= 0	Subtotal de la línea.
agregado_en	timestampz	—	No	Completo con autor y nombre	Momento de un agregado posterior.
agregado_por	text	—	No	Completo con fecha y nombre	sub que agregó la línea.

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
agregado_por_nombre	varchar	120	No	Completo con fecha y sub	Nombre copiado al agregar.

Tabla 10. Diccionario de datos de la entidad detalle_pedido. Fuente: elaboración propia a partir del esquema vigente.

Diccionario de datos — producto

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador.
nombre	varchar	120	Sí	Único; no vacío	Nombre de carta.
categoria	categoria_producto	—	Sí	Enum	Pizza, entrada, bebida, postre o extra.
precio	numeric	10,2	Sí	>= 0	Precio de venta.
disponible	boolean	—	Sí	true por defecto	Disponibilidad simple.
descripcion	varchar	200	No	No vacía si existe	Ingredientes o detalle.
imagen	varchar	80	No	Solo nombre de archivo	Recurso visual.
solo_entera	boolean	—	Sí	Solo aplicable a pizza	Impide usarla como mitad.
creado_en	timestampz	—	Sí	now()	Alta del registro.

Tabla 11. Diccionario de datos de la entidad producto. Fuente: elaboración propia a partir del esquema vigente.

Diccionario de datos — historial_estado

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	PK	Identificador.
pedido_id	integer	—	Sí	FK pedido; CASCADE	Pedido trazado.
estado	estado_pedido	—	Sí	Enum	Estado registrado.
usuario_id	text	—	Sí	No vacío	sub que ejecutó el cambio.
usuario_nombre	varchar	120	Sí	—	Nombre copiado en el momento.
fecha_hora	timestampz	—	Sí	now()	Instante del cambio.
motivo	varchar	120	Condicional	Obligatorio solo al cancelar	Causa de baja lógica.

Tabla 12. Diccionario de datos de la entidad historial_estado. Fuente: elaboración propia a partir del esquema vigente.

2.4.3 Diseño de la interfaz

La interfaz diferencia los contextos de uso. Recepción prioriza un formulario completo en computadora: cliente, modalidad, pizzas, bebidas, observación,

resumen y confirmación permanecen organizados por jerarquía. Cocina prioriza lectura rápida en celular o tableta mediante una cola ordenada, tarjetas legibles, un estado textual y una acción principal por etapa.

Las vistas que consumen datos distinguen cargando, con datos, vacío y error. Los estados no dependen solo del color; los botones expresan la transición disponible. El diseño contempla una etiqueta visual para identificar pedidos demorados cuando corresponda aplicar el criterio definido en RF-10. La Figura 5 es un prototipo de baja fidelidad elaborado desde la interfaz vigente, no una captura de ejecución.

Recepción — web, escritorio 1366 px

Max Pizzapp · Recepción

Nueva venta | Pedidos | Cerrar sesión

1 · Cliente

Nombre · celular opcional · local / para llevar

2 · Pizzas

Agregar pizza entera o mitad y mitad

Elegir sabores y extras

[+ Agregar otra pizza]

3 · Bebidas

Bebidas con cantidad

Pedido actual

Productos y cantidades

Observación para cocina

Total

[Confirmar venta]

La composición de Recepción prioriza el formulario completo en computadora. Cocina conserva botones grandes, estado textual y cola legible. Los estados cargando, con datos, vacío y error se resuelven en el mismo contenedor de cada vista.

Prototipos de baja fidelidad elaborados a partir de la interfaz vigente; no contienen datos reales de clientes.

Cocina — Android

Cocina

En vivo

2 pedidos en cocina

#44 · Ana · Para llevar

Mitad Choclo / mitad Española

Observación: sin cebolla

[Empezar]

#43 · Luis · En preparación

Pizza Peperoni + extra

Agregado recientemente

15 min · demorado

[Listo]

Mismo módulo en tableta

pedido #44

pedido #43

Figura 5. Prototipos de las pantallas principales y su adaptación. Fuente: elaboración propia a partir de la interfaz vigente.

2.4.4 Diseño de la interfaz de programación de aplicaciones

El contrato OpenAPI 3.0.3 describe nueve operaciones bajo /api/v1. La autenticación no se implementa con una ruta /auth/login: los clientes obtienen

el token mediante Keycloak y lo envían como Bearer JWT. La operación de salud es la única pública; las demás requieren identidad y rol. El archivo contrato.test.js compara las rutas declaradas con las montadas por el servidor, aunque este apartado no adelanta el resultado de su ejecución.

Método	Ruta	Auth	Rol	Entrada	2xx	Errores principales
GET	/api/v1/salud	No	Público	—	200 estado y base	503 degradado
GET	/api/v1/sesion	Bearer JWT	Recepción / Cocina	—	200 identidad y roles	401, 403
GET	/api/v1/productos	Bearer JWT	Recepción / Cocina	—	200 carta	400, 401, 403, 503
GET	/api/v1/pedidos	Bearer JWT	Recepción / Cocina	estado opcional	200 lista activa	400, 401, 403, 503
POST	/api/v1/pedidos	Bearer JWT	Recepción	pedido o venta directa	201 pedido	400, 401, 403, 409, 503
GET	/api/v1/pedidos/{id}	Bearer JWT	Recepción / Cocina	id	200 pedido e historial	400, 401, 403, 404, 503
POST	/api/v1/pedidos/{id}/lineas	Bearer JWT	Recepción	líneas y total esperado	200 pedido actualizado	400, 401, 403, 404, 409, 503
PATCH	/api/v1/pedidos/{id}/estado	Bearer JWT	Según transición	estado y versión opcional	200 pedido actualizado	400, 401, 403, 404, 409, 503
POST	/api/v1/pedidos/{id}/cancelacion	Bearer JWT	Recepción	motivo	200 pedido cancelado	400, 401, 403, 404, 409, 503

Tabla 13. Contrato vigente de la API. Fuente: elaboración propia a partir de docs/api/openapi.yaml.

RF-13 permanece especificado en el análisis de requisitos; sin embargo, al corte documental de esta versión todavía no dispone de una operación propia en el contrato OpenAPI vigente.

El contrato no expone una operación DELETE para pedidos. La cancelación se realiza mediante POST /api/v1/pedidos/{id}/cancelacion, conserva el registro y exige que el estado sea pendiente; las transiciones y los agregados usan operaciones específicas para que el servidor aplique las reglas del dominio.

2.5 Justificación del stack tecnológico

La selección tecnológica no se fundamenta en popularidad aislada, sino en su correspondencia con los requisitos y con la arquitectura adoptada. La Tabla

14 resume las versiones verificadas en la configuración y documentación vigente; su inclusión describe el diseño disponible y no reemplaza la evidencia de implementación, seguridad o pruebas de los apartados posteriores.

Componente	Versión verificada	Función	Criterio de selección vinculado al proyecto
Flutter / Dart	3.44.8 / 3.12.2	Clientes web y Android desde una base común.	Responde a RNF-04 y permite separar composiciones de Recepción y Cocina sin duplicar el dominio de interfaz.
Node.js / Express	24.15.0 LTS / 5.2.1	API y aplicación de reglas.	Centraliza validaciones de RF-02, RF-05, RF-07, RF-09 y RF-14 antes de la persistencia.
PostgreSQL	17.11	Persistencia relacional.	Sostiene las cinco entidades, restricciones, transacciones e historial exigidos por el objetivo específico 2.
Keycloak	26.7.4	Identidad, credenciales y roles.	Separa la contraseña de Max Pizzapp y aporta OIDC, tokens firmados y roles para RNF-02.
Socket.IO / socket_io_client	4.8.3 / 3.1.6	Canal bidireccional y reconexión.	Atiende RF-03, RF-04 y RF-06; ante pérdida, la persistencia permite recuperar el estado.
flutter_appauth	12.1.0	Flujo OIDC del cliente Android.	Abre el navegador del sistema y ejecuta Authorization Code + PKCE sin recoger la contraseña en la aplicación.
Caddy	2.11.4	HTTPS, archivos web y proxy inverso.	Ofrece una entrada pública cifrada y rutas controladas para API, Socket.IO y Keycloak, en apoyo de RNF-02.
Docker Engine / Compose	29.7.2 dev; 29.8.1 servidor / v5.3.1 dev; v5.5.1 servidor	Aislamiento y coordinación de servicios.	Declara API, base, identidad y proxy con redes y volúmenes explícitos; facilita reproducir el despliegue de RNF-05.
OpenAPI	3.0.3; contrato 1.0.0	Descripción verificable de la API.	Vincula diseño y rutas reales, evita endpoints genéricos y permite contrastar cliente, servidor y objetivo específico 2.

Tabla 14. Stack tecnológico y criterios de selección. Fuente: elaboración propia con base en la configuración vigente del proyecto.

La combinación mantiene responsabilidades delimitadas: Flutter presenta la interacción en web y Android; Node.js y Express concentran las reglas; PostgreSQL conserva el estado; Keycloak resuelve la identidad; Socket.IO comunica cambios; Caddy expone los servicios cifrados; Docker Compose coordina la infraestructura; y OpenAPI conserva el contrato. Esta separación permite rastrear cada decisión hacia RF o RNF concretos sin afirmar por anticipado resultados de pruebas.

2.6 Implementación

Este apartado describe lo construido hasta el corte del E3, el 3 de octubre de 2026. El desarrollo avanzó por tarjetas del tablero Kanban, de a una: cada

tarjeta tuvo su plan escrito y aprobado antes del código, commits progresivos al cerrar cada fase probada y un documento de evidencia al cerrarla. Todos los requisitos *Must* funcionan en la dirección pública, <https://maxpizzapp.tech>, contra la base de datos de producción.

2.6.1 Organización del código

El repositorio, <https://github.com/jorgeluisgutierrezrivera/max-pizzapp>, contiene el código y su documentación técnica, organizados por componente:

- **backend/**: la API en Node.js con Express, con las rutas por recurso (`src/rutas/`), la lógica de los pedidos separada de HTTP (`src/pedidos/`), el cálculo del precio (`src/precio.js`) y el canal en tiempo real (`src/tiempo-real.js`).
- **frontend/**: la app Flutter. La web arranca desde `lib/main.dart`, y el APK de Cocina, desde `lib/main_cocina.dart` y la carpeta `android/`; los dos comparten la lógica de la venta, los pedidos en vivo, los clientes de la API y del canal, y las pantallas.
- **docker/**: los archivos de Docker Compose de desarrollo y de producción, el esquema SQL y sus migraciones numeradas, el realm de Keycloak y la configuración de Caddy.
- **pruebas/**, **scripts/**, **docs/** y **.github/**: las pruebas contra Keycloak y la API reales, los scripts de publicación, de compilación del APK y de endurecimiento de Keycloak, la especificación, los planes de cada tarjeta, el contrato OpenAPI, el plan de pruebas con sus reportes y la integración continua.

Las credenciales viven en un archivo `.env` fuera del repositorio, que incluye `.env.example` con los nombres de las variables. Las migraciones de la base se

aplican en orden y nunca se edita una ya aplicada: cada cambio del modelo va en una migración nueva.

2.6.2 Avance por tarjeta

La Tabla 15 resume las tarjetas cerradas al corte del E3 y los requisitos a los que aporta cada una.

N.º	Nombre	Qué se implementó	Requisitos	Cierre
01	Esquema de datos	Las cinco entidades en PostgreSQL 17, con sus claves, restricciones y tipos enumerados	Modelo de datos (2.4.2)	22-sep
02	URL pública	Servidor VPS con Docker Compose y Caddy: HTTPS automático y la app, la API y Keycloak bajo un dominio propio	Requisito mínimo 1 · RNF-02	23-sep
03	API base	Ruta de salud y validación del token (firma, emisor, audiencia y vigencia) y del rol	RNF-02 · RNF-05	23-sep
04	App y acceso	App Flutter web con inicio de sesión por código de autorización con PKCE y la pantalla de cada rol	RF-01 · RF-08	23-sep
05	Carta y recepción	Las 15 pizzas de la carta del local, con sus precios y fotos; bebidas y extras ficticios	RF-02 (carta) · RNF-04	24-sep
06	Pedidos y cocina	Alta con el precio del servidor, cola de cocina en vivo, cambios de estado por rol, entrega, cancelación con motivo, agregados y venta directa de bebidas	RF-02 a RF-07, RF-09, RF-11, RF-14	25-sep
07	Canal en vivo y sesión	Aviso de canal caído con <i>Recargar</i> , latido de 7 s, reconexión, sesión rechazada que vuelve al acceso y corte de la conexión al vencer el token	RF-03 (CA-03.2) · RF-08 · RNF-05	30-sep
12	App Android para cocina	El mismo código Flutter como APK: acceso con el navegador del sistema, timbre por el canal de alarma, pantalla siempre encendida y publicación en el Release apk-cocina-0.1.0; instalado y probado contra producción en un Honor 50 con Android 13	RF-06 · RF-07 · RNF-04	1-oct
09	Endurecimiento de seguridad	Cabeceras y límite de peticiones en la API, ningún dato que responda 500, Keycloak endurecido con un script, CSP impuesta y SSH solo con llave	RNF-02 · requisitos mínimos 2, 7 y 8	2-oct
10	Pruebas con evidencia	Reportes versionados, integración continua, RNF-01 medido en producción y la tabla de casos del 2.8	RNF-01, RNF-03, RNF-04 y RNF-05	2-oct

Tabla 15. Tarjetas implementadas al corte del E3. Fuente: elaboración propia.

La tarjeta 08, que implementa RF-13 (marcar un producto como agotado), queda para la Semana 4. La tarjeta 12, incorporada al alcance en la tutoría T3 del 28 de septiembre (D-43), se cerró con la publicación del Release apk-cocina-0.1.0. El APK se instaló y probó contra producción en un Honor 50 con

Android 13. Android 7 es la versión mínima configurada; no se presenta como el extremo inferior de un intervalo completamente probado. La ampliación desde el perfil solo web del E1 se registra como una desviación de alcance.

2.6.3 El flujo implementado: del mostrador a la cocina

El flujo *Must have* completo recorre la pantalla, la API, la base de datos y el canal en tiempo real:

1. Recepción arma la venta en un único formulario: el cliente, si es para comer en el local o para llevar, las pizzas (cada una en un cuadro propio, entera o de dos mitades, con sus extras), las bebidas y una observación para Cocina.
2. La app envía `POST /api/v1/pedidos` con las líneas y el total que mostró la pantalla. El servidor comprueba el rol, recalcula el precio con la carta de la base y, si coincide, guarda en una sola transacción el cliente, el pedido con su número del día, sus líneas y la primera fila del historial.
3. Confirmada la transacción, el servidor emite `pedido:nuevo` por el canal: Cocina lo recibe sin el celular del cliente, y Recepción, completo.
4. Cocina ve aparecer el pedido al final de su cola, con un timbre, y lo pasa a *en preparación* y luego a *listo*.
5. Recepción recibe el aviso de *listo* (sonido, mensaje y contador en la pestaña) y marca el pedido como entregado.
6. Si el canal en vivo se corta, las dos pantallas lo indican con una banda en menos de 10 segundos y ofrecen *Recargar*, que vuelve a leer el estado desde la API. Mientras tanto se sigue trabajando, porque las acciones van por la API y no por el canal.

La Tabla 16 muestra cómo se corresponden las operaciones CRUD sobre el pedido con el contrato de la Tabla 13.

Operación	Ruta	Qué hace
Crear	POST /api/v1/pedidos	Registra el pedido con el precio calculado por el servidor y su número del día; también la venta directa de bebidas
Listar	GET /api/v1/pedidos	Devuelve los pedidos activos, o los de los estados indicados; Cocina los recibe sin el celular del cliente
Consultar	GET /api/v1/pedidos/{id}	Devuelve el pedido con sus líneas y su historial de estados
Editar	PATCH /api/v1/pedidos/{id}/estado POST /api/v1/pedidos/{id}/lineas	Avanza el estado según el rol, y agrega productos a un pedido ya enviado
Dar de baja	POST /api/v1/pedidos/{id}/cancelacion	Cancela con motivo un pedido pendiente; el pedido no se borra y el historial registra quién, cuándo y por qué

Tabla 16. Operaciones CRUD sobre el pedido. Fuente: elaboración propia.

2.6.4 Decisiones de implementación

- **El precio lo calcula el servidor.** La app envía qué se vende y el total que mostró; el servidor lo recalcula con la carta de la base, en centavos enteros. Si no coincide, responde 409 y no guarda nada: una app modificada no puede fijar el precio.
- **Una transacción por operación, y el aviso después de confirmarla.** Si la base falla a mitad de camino, no queda nada a medias. El evento se emite solo después del `COMMIT`, para que ninguna pantalla muestre un pedido que no llegó a existir.
- **Cada cambio de estado tiene un rol dueño.** Una tabla de transiciones (Figura 8) define desde qué estado vale cada cambio y qué rol lo hace. El estado se lee con `SELECT ... FOR UPDATE`: si dos dispositivos cambian el mismo pedido a la vez, el segundo espera, ve el cambio del primero y recibe un 409.
- **La versión del pedido.** El cambio de estado lleva la versión que vio la pantalla, que es la cantidad de líneas del pedido. Si Recepción agregó algo justo antes, el servidor responde 409 y la pantalla vuelve a leer: Cocina no marca listo un pedido sin haber visto lo agregado.

- **El número del día.** Cada pedido recibe un número que empieza en 1 cada día, en la hora de Bolivia, para cantarlo en el mostrador. Se asigna después de un bloqueo de transacción (`pg_advisory_xact_lock`) y lo garantiza una restricción de unicidad.
- Un latido corto en el canal (D-44). Con los valores de fábrica de Socket.IO, una conexión colgada sin aviso tardaba hasta 45 s en notarse. El servidor envía un latido cada 4 s y da la conexión por perdida a los 3 s sin respuesta. La configuración establece un máximo teórico de 7 s para detectar una pérdida silenciosa y una espera aproximada de 1 s antes de mostrar la banda. En las pruebas formales, el peor tiempo de detección observado fue de 6,89 s.
- **Ninguna conexión dura más que su token (D-47).** El token se comprueba al conectarse; sin otra medida, una conexión seguiría recibiendo pedidos después de vencido. El servidor la corta a la hora exacta del vencimiento, y la app, que renueva el token a los 48 minutos, se reconecta con el nuevo.
- **El APK reutiliza la app web (D-48 a D-51).** Solo tres piezas dependen de la plataforma: el inicio de sesión, que en Android abre el navegador del sistema con AppAuth y PKCE, de modo que la contraseña nunca pasa por la app; el timbre, que suena por el canal de alarma desde que se abre la cola; y la pantalla siempre encendida mientras la cola está abierta.

2.6.5 Interfaz implementada

La toma de pedido se implementó como un único formulario en patrón F (Figura 6), a partir de la observación de la tutoría del 24 de septiembre: el cliente y la modalidad arriba, las pizzas y las bebidas debajo, y el pedido armado, con la observación y la confirmación, siempre a la vista a la derecha.

Cada pizza se arma en un cuadro propio, entera o de dos mitades, con sus extras y su cantidad.

Recepción Nueva venta Pedidos 🔊 🔒 Cerrar sesión

1 - Cliente Vender bebidas

Nombre del cliente: Juan España Celular (opcional): 50000001

¿Para comer aquí o para llevar?

☐ Para comer aquí ☒ Para llevar

2 - Pizzas

1 Cuatro quesos + Extra queso Bs 63

+ Agregar otra pizza

3 - Bebidas

Agua mineral 600 ml Bs 6

Gaseosa 2 L Bs 18 1

Jugo natural 1 L Bs 15

Pedido
Juan España - 50000001
- Para llevar

1 x Cuatro quesos + Extra queso Bs 63
1 x Gaseosa 2 L Bs 18

Observación para cocina: Bien cocida

Total Bs 81

> Confirmar venta · Bs 81
Cancelar venta

Figura 6. Formulario de venta de Recepción, con una venta lista para confirmar. Fuente: captura del sistema desplegado en Edge, 02/10/2026.

Cocina utiliza el APK Android derivado de la misma base Flutter. La validación física documentada se realizó en un Honor 50 con Android 13. La interfaz presenta una cola de tarjetas grandes en orden de llegada (Figura 7). Cada pedido nuevo entra con un timbre y la marca *Nuevo*, y un solo botón por tarjeta lo lleva al paso siguiente: *Empezar* y luego *Listo*. Si se pierde la conexión en vivo, una banda lo dice y ofrece *Recargar*.

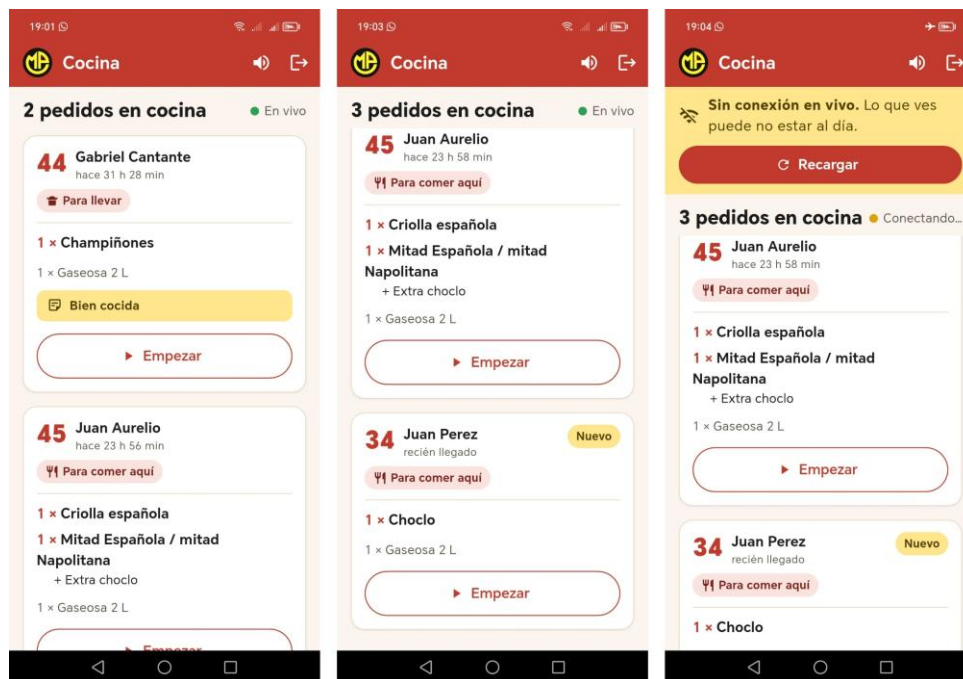


Figura 7. El APK de Cocina en un teléfono Android: la cola en vivo (izquierda), un pedido que llega solo (centro) y el aviso de canal caído con Recargar (derecha). Fuente: captura del sistema desplegado, 01/10/2026.

Recepción sigue sus pedidos en la pestaña *Pedidos*, donde cada tarjeta ofrece solo las acciones que su estado permite: cancelar mientras el pedido está pendiente; agregar pizzas y bebidas cuando está pendiente o en preparación, y solo bebidas cuando está listo; las líneas previamente enviadas no se modifican ni eliminan, sino que se incorporan nuevas líneas; y entregar o llamar al cliente cuando está listo. Cada vista que consume datos implementa los cuatro estados (cargando, con datos, vacío y error, con la opción de reintentar), y las pruebas automáticas verifican el contraste de cada combinación de colores con la fórmula de WCAG.

2.6.6 Fragmentos de código ilustrativos

Quién puede hacer cada cambio de estado, y desde qué estado, vive en una sola tabla del servidor:

```
const TRANSICIONES = {
  en_preparacion: { desde: ['pendiente'], rol: 'cocina' },
  listo: { desde: ['en_preparacion'], rol: 'cocina' },
  entregado: { desde: ['listo'], rol: 'recepcion' },
  cancelado: { desde: ['pendiente'], rol: 'recepcion' },
};
```

Figura 8. Las transiciones de estado y el rol que hace cada una. Fuente: backend/src/pedidos/estados.js.

El canal en vivo declara su latido y corta cada conexión a la hora en que vence su token:

```
const LATIDO = { pingInterval: 4000, pingTimeout: 3000 };

io.on('connection', (socket) => {
  const { usuario } = socket.data;
  for (const rol of usuario.roles) socket.join(SALA[rol]);
  if (Number.isFinite(usuario.venceEn)) {
    const restante = Math.min(
      Math.max(usuario.venceEn - Date.now(), 0), PLAZO_MAXIMO_MS);
    const corte = reloj.setTimeout(
      () => socket.disconnect(true), restante);
    socket.on('disconnect', () => reloj.clearTimeout(corte));
  }
});
```

Figura 9. El latido del canal y el corte de la conexión al vencer el token (fragmento). Fuente: backend/src/tiempo-real.js.

2.6.7 Dificultades y su resolución

- **La API se caía al recibir el canal en tiempo real.** Express y Socket.IO respondían la misma petición, porque Express se agregaba al servidor después del canal. Las pruebas no lo detectaban porque montaban el canal sin Express. El servidor pasó a armarse en un solo lugar, que usan el arranque y las pruebas.
- **El aviso de un producto agregado no llegaba a Cocina.** Lo detectó la prueba del autor: el aviso existía en el canal, pero no en el puente que conecta la API con él. Las pruebas pasaron a avisar por el mismo camino que la API.

- Una conexión colgada tardaba hasta 45 s en notarse. El corte silencioso de la red no cierra la conexión y solo lo detecta el latido. Se acortó el latido (D-44) y se midió con un tapón que interrumpe la red sin aviso: la configuración establece un máximo teórico de 7 s para detectar una pérdida silenciosa y una espera aproximada de 1 s antes de mostrar la banda. En las pruebas formales, el peor tiempo de detección observado fue de 6,89 s.
- **Dos errores 500 que se podían provocar sin token.** Un nombre con el carácter nulo llegaba a la base, que lo rechazaba, y un cuerpo de más de 100 KB caía en el manejador general. Se confirmaron contra la API real antes de corregirlos, y ahora responden 400, 413 y 415.
- **El realm no se importaba desde cero.** La descripción de un cliente tenía 266 caracteres y Keycloak admite 255. En producción no se notaba, porque el realm se importa una sola vez; en un clon nuevo, como el del tribunal, no habría arrancado. Se acortó, y la importación desde cero quedó probada.

2.7 Seguridad

La seguridad de Max Pizzapp se apoya en tres capas: el proveedor de identidad, que autentica y administra las credenciales; el servidor de aplicación, que autoriza cada operación y valida cada dato; y la infraestructura, que cifra el tráfico y expone solo lo necesario. Los controles principales se verificaron mediante pruebas funcionales, pruebas de seguridad y evidencias de configuración; sus resultados se relacionan con los casos del apartado 2.8.

2.7.1 Autenticación y gestión de sesiones

La autenticación la realiza Keycloak 26.7.4 con OpenID Connect, mediante el flujo de código de autorización con PKCE (Sakimura, Bradley y Agarwal, 2015). La contraseña se escribe en la pantalla de Keycloak y nunca pasa por

Max Pizzapp. En el APK, el acceso abre el navegador del sistema, como recomienda la práctica para aplicaciones nativas (Denniss y Bradley, 2017). Keycloak entrega un token de acceso JWT firmado (Jones, Bradley y Sakimura, 2015) con una vigencia de 60 minutos, como exige RNF-02; la app lo renueva a los 48 minutos, antes de que venza.

El servidor rechaza con 401 un token ausente, con la firma alterada, de otro emisor, de otra audiencia o vencido. Si un 401 persiste después de renovar, la sesión termina y la app vuelve a la pantalla de acceso. Cerrar la sesión descarta el token, para que otro compañero inicie la suya en el mismo dispositivo (RF-08). Keycloak bloquea la cuenta durante un tiempo tras cinco intentos fallidos, y también ante intentos demasiado rápidos, y exige contraseñas de al menos 12 caracteres.

2.7.2 Autorización por rol

Cada ruta protegida verifica el rol en el servidor, antes de consultar la base: la ocultación de un botón en la pantalla no autoriza nada. Las rutas que cambian un pedido comprueban además la transición (Figura 8): Cocina empieza y marca listo; Recepción vende, agrega, entrega y cancela. Un rol equivocado recibe 403 `ROL_SIN_PERMISO`. El canal en vivo autentica con el mismo token y reparte los avisos por sala de rol, de modo que Cocina recibe los pedidos sin el celular del cliente.

Las cuentas nuevas nacen con el menor privilegio posible: Keycloak no les asigna ningún rol del sistema, y la API les responde 403 en todas las rutas hasta que se les asigne uno (D-54). La configuración de seguridad de Keycloak se aplica con un script versionado en el repositorio, igual en desarrollo y en producción (D-53), y acepta solo cuatro direcciones de retorno exactas.

2.7.3 Validación de entradas y vulnerabilidades habituales

La validación es doble. La app valida para avisar a tiempo; la API vuelve a validar todo, porque es la única validación confiable; y la base tiene la última palabra con sus restricciones. Todas las respuestas de error comparten el formato `{ "error": { "codigo", "mensaje" } }` y ninguna expone trazas internas. La Tabla 17 resume la validación de los datos principales.

Dato	En la app	En la API y en la base	Si no cumple
Nombre del cliente	Obligatorio; hasta 120 caracteres	Obligatorio, hasta 120, sin caracteres de control; restricción de no vacío	400 VENTA_INVALIDA
Celular	Opcional; 8 dígitos que empiezan con 5, 6 o 7	Lo mismo; restricción <code>^[5-7][0-9]{7}\$</code> y único	400 VENTA_INVALIDA
Líneas, mitades y extras	Se eligen de la carta; al menos una pizza	Que existan y estén disponibles; una pizza <i>solo entera</i> no va por mitades; claves foráneas	400; 409 PRODUCTO_NO_DISPONIBLE
Cantidad	De 1 a 999, con botones	Entero de 1 a 999; restricción en la base	400 VENTA_INVALIDA
Total	Calculado con la misma regla	Recalculado por el servidor; si no coincide, no se guarda nada	409 PRECIO_CAMBIADO
Estado y motivo	Solo el botón que corresponde al rol; motivo obligatorio para cancelar	Rol, transición, versión del pedido y motivo de hasta 120 caracteres	400; 403; 409
Cuerpo de la petición	Lo arma la app	JSON en UTF-8 de hasta 100 KB	400; 413; 415

Tabla 17. Validación doble de los datos principales. Fuente: elaboración propia a partir del plan de la tarjeta 09.

La Tabla 18 contrasta el sistema con las diez categorías de riesgo del OWASP Top 10:2025, la referencia habitual de vulnerabilidades en aplicaciones web (OWASP Foundation, 2025).

Categoría	Tratamiento en Max Pizzapp
A01 — Pérdida de Control de Acceso	Token y rol verificados en el servidor en cada ruta; transiciones por rol; cuentas nuevas sin rol (403); Cocina no recibe el celular del cliente.
A02 — Configuración de Seguridad Incorrecta	Cabeceras de seguridad con Helmet; CSP impuesta; la API no anuncia su tecnología; Keycloak configurado por script. La base de datos y los servicios internos no se exponen directamente a Internet; el acceso público se concentra en los servicios necesarios para la solución.
A03 — Fallas en la Cadena de Suministro de Software	Dependencias exactas con <code>package-lock.json</code> y <code>pubspec.lock</code> ; npm audit sin vulnerabilidades; versiones fijadas de cada imagen; ningún script de terceros en la app.
A04 — Fallas Criptográficas	HTTPS con HSTS en todo el tráfico público y WSS en el canal; contraseñas con Argon2id; tokens firmados, verificados con las claves públicas del emisor.

Categoría	Tratamiento en Max Pizzapp
A05 — Inyección	Consultas parametrizadas, sin concatenar texto del usuario en SQL; los textos se muestran sin interpretar HTML; caracteres de control rechazados.
A06 — Diseño Inseguro	El precio lo decide el servidor; las reglas del negocio viven en el servidor y en la base; la versión del pedido evita decisiones sobre datos que no se vieron.
A07 — Fallas de Autenticación	Keycloak con OpenID Connect y PKCE; bloqueo tras cinco intentos; contraseñas de 12 caracteres o más; token de 60 minutos.
A08 — Fallas en la Integridad del Software o de los Datos	Transacciones en cada operación; el aviso en vivo solo después del COMMIT; migraciones numeradas que no se editan; las pruebas corren en cada push.
A09 — Fallas en el Registro, Alerta y Monitoreo de Seguridad	Eventos de acceso de Keycloak durante 7 días; historial de cada pedido con quién, cuándo y por qué; monitor de disponibilidad con aviso por correo.
A10 — Manejo Inadecuado de Condiciones Excepcionales	Ningún dato de entrada hace responder 500: 400, 413 o 415 en el formato único; 429 pasadas las 600 peticiones por minuto de una IP.

Tabla 18. Riesgos del OWASP Top 10:2025 y su tratamiento. Fuente: elaboración propia con base en OWASP Foundation (2025).

Para limitar el abuso, la API acepta hasta 600 peticiones por minuto por IP y responde 429 a partir de la siguiente. El límite es generoso a propósito: todo el local sale a Internet por una sola IP, y un límite estricto lo castigaría entero (D-52). La app web aplica una política de seguridad de contenido acorde con la configuración publicada (W3C, s. f.). Los scripts proceden del propio dominio y script-src agrega wasm-unsafe-eval por CanvasKit; style-src permite estilos en línea; font-src incorpora fonts.gstatic.com; connect-src admite el propio origen, wss://maxpizzapp.tech, https://auth.maxpizzapp.tech y fonts.gstatic.com; e img-src admite el propio origen, data: y blob:. object-src se mantiene en none, base-uri y form-action en el propio origen, y frame-ancestors en none. La política se midió primero en modo informe y después se impuso (D-55). La API no habilita CORS: la app y la API comparten origen y no lo necesitan.

2.7.4 Cifrado de credenciales y comunicaciones

Caddy termina TLS con certificados que se renuevan solos y agrega la cabecera HSTS, que obliga al navegador a usar siempre HTTPS con el dominio (Hodges, Jackson y Barth, 2012). El canal en vivo viaja por WebSocket seguro (WSS). Las contraseñas no se guardan en Max Pizzapp:

Keycloak las conserva con Argon2id, una función de *hash* costosa en memoria a propósito, que encarece los ataques por fuerza bruta (Biryukov et al., 2021). La API verifica la firma de cada token con las claves públicas que publica Keycloak (JWKS). La base de datos no se expone fuera de la red interna de Docker, y el servidor acepta SSH solo con llave.

2.7.5 Gestión de variables sensibles

Las credenciales de la base, de Keycloak y de las cuentas de prueba viven en el archivo `.env` del servidor, que nunca entra al repositorio; `.env.example` documenta los nombres de las variables sin sus valores. Antes del E3 se revisaron el árbol de trabajo y el historial completo del repositorio utilizando los valores sensibles conocidos como patrones de búsqueda, sin encontrar los secretos operativos versionados. La integración continua corre sin ningún secreto, y los reportes de las pruebas se revisaron para que no contengan la contraseña de prueba ni ningún token. Este documento no contiene credenciales: las de las cuentas de prueba se entregan por separado, en el campo de texto de la tarea.

2.8 Pruebas

El plan de pruebas vive en el repositorio, en `docs/pruebas/README.md`, junto con la tabla de casos de la que sale este apartado y los reportes de cada ejecución, en `docs/pruebas/reportes/`. Cada reporte lleva adentro la fecha, la versión y el entorno en que se corrió, y el historial de Git conserva los anteriores.

2.8.1 Plan de pruebas

Las pruebas se organizan en niveles, de los que no necesitan nada para correr a los que miden el sistema desplegado (Tabla 19).

Nivel	Qué prueba	Herramienta	Alcance	Reporte
API	Rutas, token y rol, validación, precio, estados, canal en vivo y contrato OpenAPI, con la base simulada	node:test (Node 24.15)	288 pruebas	api.txt, api-junit.xml
App	Venta, pedidos, cocina, acceso en la web y en el APK, sesión, canal y sus cortes, avisos, anchos y contraste, con la API simulada	flutter_test (Flutter 3.44.8)	289 pruebas	app.txt
Punta a punta	Acceso por PKCE, carta, pedidos y matriz de errores, con las cuentas de prueba contra el entorno real	Python 3	5 sondas	sonda-*.txt
Tiempo real	Del cambio a la llegada del aviso en la otra pantalla	socket.io-client	30 repeticiones	tiempo-real.txt
Carga	El listado con 50 pedidos activos y 5 usuarios a la vez	k6 2.3.0, en Docker	60 s	carga-k6.txt
Disponibilidad	GET /api/v1/salud, las 24 horas	UptimeRobot	Cada 5 min desde el 23-sep	Panel del monitor
Integración continua	Las dos suites, en un clon limpio	GitHub Actions	Cada push	Corridas en el repositorio

Tabla 19. Niveles de prueba. Fuente: elaboración propia a partir de docs/pruebas/README.md.

Las pruebas de la API y de la app no necesitan base de datos, Keycloak ni archivo `.env`: la base se simula y anota cada consulta, y los tokens los firma un emisor local. Las sondas, la medición del tiempo real y la carga entran con las cuentas de prueba y crean pedidos con datos ficticios, que cierran al terminar.

2.8.2 Pruebas automatizadas

La suite de la API se ejecuta con `cd backend && npm ci && npm test`, sin más requisito que Node, y deja los reportes `docs/pruebas/reportes/api.txt` y `docs/pruebas/reportes/api-junit.xml` (Node.js, s. f.). La suite de la app se ejecuta con `flutter test` (Flutter, 2026c). Las dos corren además en GitHub Actions en cada push a la rama principal: una máquina limpia, sin `.env`, sin base y sin Keycloak, instala las dependencias exactas y ejecuta ambas suites y el análisis estático de la app (GitHub, s. f.). Al corte del 2 de octubre se verificaron cinco ejecuciones consecutivas del flujo de integración continua «Pruebas», todas finalizadas correctamente. La quinta correspondió al push de cierre posterior a la redacción inicial de la tarjeta 10. Estas ejecuciones

demuestran que el comando de la entrega funciona en un clon limpio. Las pruebas contra producción no corren allí, porque necesitan la contraseña de las cuentas de prueba, que no se guarda en GitHub.

2.8.3 Casos de prueba

La Tabla 20 reúne el camino feliz y el error de cada requisito *Must* (CP-01 a CP-14), los casos de seguridad (CP-15 a CP-22) y una fila por cada medida de los requisitos no funcionales (CP-23 a CP-28). La columna «Resultado obtenido» indica, en cada fila, dónde está su evidencia: un reporte de docs/pruebas/reportes/, una prueba automática, una figura o un commit.

ID	Escenario evaluado	Resultado esperado	Resultado obtenido y evidencia	Estado
CP-01	CA-01.1 · Una cuenta con credenciales válidas inicia sesión	Recibe su token con su rol y 60 min de vigencia, y entra a su pantalla	Las dos cuentas, con su rol y 60 min. Evidencia: sonda-acceso.txt; pruebas de la app del acceso	Aprobado
CP-02	CA-01.2 · Se ingresa una contraseña equivocada	Keycloak no entrega código ni token	Credenciales rechazadas. Evidencia: sonda-errores.txt	Aprobado
CP-03	CA-02.1 · Recepción vende productos disponibles y confirma	El pedido se crea pendiente, con su número del día y el precio del servidor	201, pendiente, Bs 189 y número del día. Evidencia: sonda-pedidos.txt; prueba de la app de la venta completa	Aprobado
CP-04	CA-02.2 · Se intenta confirmar una venta sin productos	No se crea; se pide agregar al menos un producto	La app no deja confirmar; la API responde 400. Evidencia: sonda-errores.txt	Aprobado
CP-05	CA-03.1 · Cocina cambia el estado de un pedido	Recepción lo ve en menos de 2 s, sin recargar	Peor caso de 262 ms en 30 repeticiones, en producción. Evidencia: tiempo-real.txt	Aprobado
CP-06	CA-03.2 · Se cae el canal en vivo	Aviso visible y opción de recargar el estado	En el APK, la banda apareció aproximadamente a los 3 s al activar el modo avión. En la prueba formal de pérdida silenciosa, el peor tiempo de detección fue de 6,89 s y el aviso visual apareció aproximadamente un segundo después, dentro del límite de 10 s. Recargar relee el estado. Evidencia: pruebas de la app del canal; Figura 7	Aprobado
CP-07	CA-04.1 · Cocina marca listo un pedido	Recepción recibe un aviso visible	Suena, dice quién lo marcó y lo cuenta en la pestaña. Evidencia: prueba de la app del aviso de listo	Aprobado
CP-08	CA-04.2 · No hay pedidos listos	Pantalla vacía, sin avisos pendientes	«No hay pedidos por atender». Evidencia: prueba de la app del estado vacío	Aprobado

ID	Escenario evaluado	Resultado esperado	Resultado obtenido y evidencia	Estado
CP-09	CA-05.1 · Recepción entrega un pedido listo	Pasa a entregado y sale de la lista	200, entregado, con el historial de los cuatro pasos. Evidencia: sonda-pedidos.txt	Aprobado
CP-10	CA-05.2 · Recepción intenta entregar un pedido que no está listo	La operación se rechaza	409. Evidencia: sonda-pedidos.txt; prueba de la API de las transiciones	Aprobado
CP-11	CA-06.1 · Recepción confirma un pedido nuevo	Aparece en Cocina en menos de 2 s, al final de la cola	Al final y marcado <i>Nuevo</i> ; peor caso de 303 ms en 30 repeticiones. Evidencia: tiempo-real.txt	Aprobado
CP-12	CA-06.2 · No hay pedidos pendientes	Cocina ve la pantalla vacía	«No hay pedidos en cocina». Evidencia: prueba de la app del estado vacío	Aprobado
CP-13	CA-07.1 · Cocina empieza un pedido y lo marca listo	Pendiente → en preparación → listo, con cada cambio guardado	200 y 200, con quién hizo cada paso. Evidencia: sonda-pedidos.txt	Aprobado
CP-14	CA-07.2 · Cocina intenta cambiar un pedido entregado	Se rechaza por ser un estado final	409. Evidencia: prueba de la API de las transiciones	Aprobado
CP-15	Una ruta protegida sin token	401	401 TOKEN_AUSENTE. Evidencia: sonda-errores.txt	Aprobado
CP-16	Un token con la firma alterada	401	401 TOKEN_INVALIDO. Evidencia: sonda-errores.txt	Aprobado
CP-17	Cocina intenta vender, cancelar o agregar	403	403 ROL_SIN_PERMISO en los tres. Evidencia: sonda-errores.txt	Aprobado
CP-18	Recepción intenta empezar un pedido	403	403 ROL_SIN_PERMISO. Evidencia: sonda-pedidos.txt	Aprobado
CP-19	Una cuenta nueva, sin rol	403 en todas las rutas	403 ROL_SIN_PERMISO. Evidencia: commit f8072da; pruebas de la API y de la app	Aprobado
CP-20	Datos inválidos: celular, carácter nulo, cantidad, filtro, número de pedido, motivo y cuerpo que no es JSON	400 en el formato único	400 con su código en cada caso. Evidencia: sonda-errores.txt	Aprobado
CP-21	Un cuerpo de 150 KB o con otro juego de caracteres	413 o 415	413 y 415. Evidencia: sonda-errores.txt	Aprobado
CP-22	Más de 600 peticiones por minuto desde una IP	429 a partir de la 601	600 respondieron y 100 recibieron 429. Evidencia: commit fb19f9c; prueba de la API del límite	Aprobado
CP-23	RNF-01 · Propagación del aviso en vivo	Peor caso de 30 repeticiones < 2 s	303, 262 y 705 ms, según el aviso (Tabla 21). Evidencia: tiempo-real.txt	Aprobado
CP-24	RNF-01 · Listado con 50 activos y 5 usuarios	Percentil 95 < 2 s	296 ms en 235 lecturas, ninguna fallida. Evidencia: carga-k6.txt	Aprobado
CP-25	RNF-02 · Identidad, token, cifrado y rol	Keycloak, token de 60 min, HTTPS, Argon2id y rol en el servidor	Verificados (2.7). Evidencia: plan de la tarjeta 09, sección 9	Aprobado

ID	Escenario evaluado	Resultado esperado	Resultado obtenido y evidencia	Estado
CP-26	RNF-03 · Pasos y estados	Venta en 3 pasos o menos; 4 estados en cada vista con datos	Tres pasos sin cambiar de pantalla; 12 de 12 estados. Evidencia: pruebas de la app	Aprobado
CP-27	RNF-04 · Web adaptable y APK Android	Web sin desplazamiento horizontal a 1366 y 768 px, en Chrome y Edge; APK con Android 7 como mínimo configurado	Las tres vistas fueron verificadas sin desplazamiento horizontal no previsto a 1366 × 768 y 768 × 1024. La evidencia manual documentada corresponde a Edge 154 y existen recorridos previos en Chrome 154. El APK fue validado en un Honor 50 con Android 13. Queda pendiente completar la matriz de las dos versiones requeridas de Chrome y Edge y ampliar la evidencia móvil a otro tamaño de pantalla.	Parcial
CP-28	RNF-05 · Disponibilidad y canal caído	≥ 99 % de respuestas 200 en 7 días o más; aviso < 10 s	UptimeRobot registró 100 % de disponibilidad durante 9 días y 20 horas, sin incidentes. En diez cortes medidos de pérdida silenciosa del canal, el peor tiempo de detección fue de 6,89 s y el aviso visual apareció aproximadamente un segundo después, manteniéndose dentro del límite de 10 s. Evidencia: monitor de UptimeRobot y pruebas del canal	Aprobado

Tabla 20. Casos de prueba y resultados. Fuente: elaboración propia a partir de docs/pruebas/README.md.

2.8.4 Requisitos no funcionales

RNF-01. Las mediciones se corrieron el 2 de octubre de 2026 contra producción, con las cuentas de prueba (Tabla 21). La propagación se midió con un script propio sobre `socket.io-client`, que registra el instante de la petición y el de la llegada del evento en la otra pantalla. La carga, con k6, cuyo umbral de 2 s está escrito en el propio script (Grafana Labs, s. f.): con 50 pedidos activos, 5 usuarios a la vez, tres de Recepción y dos de Cocina, leyeron el listado una vez por segundo durante 60 s. Los 50 pedidos se crearon al empezar y se cancelaron al terminar.

Medición	Muestras	Mínimo	Mediana	p95	Máximo
Pedido nuevo → Cocina	30	233 ms	236 ms	290 ms	303 ms
Cambio de estado → Recepción	30	227 ms	231 ms	258 ms	262 ms
Lo agregado → Cocina	30	230 ms	239 ms	247 ms	705 ms

Medición	Muestras	Mínimo	Mediana	p95	Máximo
Listado con 50 activos y 5 usuarios (k6)	235	273 ms	286 ms	296 ms	369 ms

Tabla 21. Mediciones de RNF-01 en producción, 02/10/2026. Fuente: docs/pruebas/reportes/tiempo-real.txt y carga-k6.txt.

El peor caso observado fue de 705 ms y se mantuvo por debajo del límite de 2 s. Las mediciones muestran un piso próximo a 230 ms; la prueba no aisló cuánto corresponde a red, procesamiento del servidor o propagación del evento.

RNF-03. Un paso es una sección del formulario de venta: el cliente, los productos y confirmar (D-60). Un pedido exige solo el cliente, al menos una pizza y confirmar; las bebidas y la observación son opcionales. Una prueba de la app hace una venta completa en esos tres pasos y comprueba que ninguno lleva a otra pantalla. Los cuatro estados se comprobaron en las tres vistas que consumen datos (la venta, los pedidos de Recepción y la cola de Cocina): 12 de 12, cada uno con su prueba.

RNF-04. Las pruebas de la app recorren las tres vistas a 1366 × 768 y a 768 × 1024 con el contenido que más ocupa, y comprueban que nada se desborda, que ningún texto visible queda fuera de la pantalla y que nada se desplaza de lado. La comprobación se calibró antes de usarla, mostrándole a propósito tres defectos que rechazó. Además, el autor recorrió el ciclo completo de un pedido en producción en Microsoft Edge 154 a esos dos tamaños (Figura 10); Chrome 154 corresponde a las pruebas del autor de las tarjetas anteriores. Para Cocina, la misma base Flutter se compiló como el APK del Release apk-cocina-0.1.0: Android 7 es la versión mínima configurada y la validación física se realizó contra producción en un Honor 50 con Android 13. No se infiere compatibilidad probada con otras versiones o modelos.

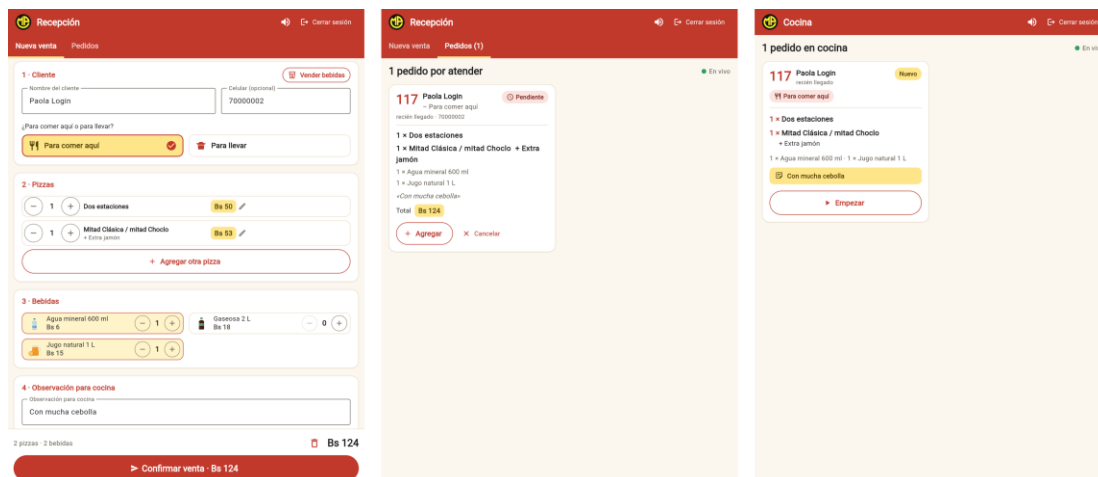


Figura 10. La venta, los pedidos de Recepción y la cola de Cocina a 768 × 1024 px, en Edge 154. Fuente: captura del sistema desplegado, 02/10/2026.

RNF-05. El monitor de UptimeRobot pide `GET /api/v1/salud` cada 5 minutos, las 24 horas, desde el 23 de septiembre de 2026 a la 01:19. Al 2 de octubre a las 21:48, tras 9 días y 20 horas, registra el 100 % de disponibilidad, sin ningún incidente. Un intervalo de cinco minutos podría no registrar una interrupción más breve que dicho periodo. La otra mitad del requisito, el aviso de canal caído en menos de 10 s, se verificó en CP-06.

2.8.5 Alcance y límites de las pruebas

No se automatizaron pruebas de punta a punta con un navegador (Playwright o Selenium): Flutter dibuja la interfaz en un lienzo y no en elementos HTML, lo que vuelve frágiles esas pruebas. El recorrido completo lo cubren las sondas contra producción, las pruebas de la app y las pruebas del autor en la web y en el APK. Tampoco se midió la cobertura de código: un porcentaje no indica qué requisito está probado, y la tabla de casos sí. Las mediciones de RNF-01 corresponden a la carga que fija el requisito para un local piloto, no a pruebas de estrés.

2.9 Despliegue

Este apartado se completará en el E4 con la plataforma utilizada, el procedimiento de despliegue, la configuración de entornos y los accesos públicos de la solución.

CAPÍTULO 3

CAPÍTULO 3. CONCLUSIONES Y RECOMENDACIONES

3.1 Conclusiones

Este apartado se completa en el E4, con una conclusión por cada objetivo específico, su grado de cumplimiento y la evidencia que lo respalda.

3.2 Recomendaciones

Se completa en el E4.

BIBLIOGRAFÍA CONSULTADA

BASS, Len; CLEMENTS, Paul y KAZMAN, Rick. Software Architecture in Practice. 4.^a ed. Boston: Addison-Wesley Professional, 2021. ISBN: 978-0-13-688609-9.

BIRYUKOV, Alex; DINU, Daniel; KHOVRATOVICH, Dmitry y JOSEFSSON, Simon. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. RFC 9106 [en línea]. IETF, 2021. DOI: 10.17487/RFC9106. Disponible en: <https://www.rfc-editor.org/info/rfc9106> [consulta: 2 de octubre de 2026].

CODD, Edgar F. A relational model of data for large shared data banks. Communications of the ACM [en línea]. 1970, vol. 13, n.º 6, pp. 377-387. DOI: 10.1145/362384.362685. Disponible en: <https://doi.org/10.1145/362384.362685> [consulta: 29 de septiembre de 2026].

DENNISS, William y BRADLEY, John. OAuth 2.0 for Native Apps. RFC 8252 [en línea]. IETF, 2017. DOI: 10.17487/RFC8252. Disponible en: <https://www.rfc-editor.org/info/rfc8252> [consulta: 2 de octubre de 2026].

DOCKER. How Compose works [en línea]. [s. f.a]. Disponible en: <https://docs.docker.com/compose/intro/compose-application-model/> [consulta: 30 de septiembre de 2026].

DOCKER. What is a container? [en línea]. [s. f.b]. Disponible en: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/> [consulta: 30 de septiembre de 2026].

DUSSEAULT, Lisa y SNELL, James. PATCH Method for HTTP. RFC 5789 [en línea]. IETF, 2010. DOI: 10.17487/RFC5789. Disponible en: <https://www.rfc-editor.org/info/rfc5789> [consulta: 29 de septiembre de 2026].

FETTE, Ian y MELNIKOV, Alexey. The WebSocket Protocol. RFC 6455 [en línea]. IETF, 2011. DOI: 10.17487/RFC6455. Disponible en: <https://www.rfc-editor.org/info/rfc6455> [consulta: 29 de septiembre de 2026].

FIELDING, Roy T.; NOTTINGHAM, Mark y RESCHKE, Julian. HTTP Semantics. RFC 9110 [en línea]. IETF, 2022. DOI: 10.17487/RFC9110. Disponible en: <https://www.rfc-editor.org/info/rfc9110> [consulta: 29 de septiembre de 2026].

FLUTTER. General approach to adaptive apps [en línea]. 2026a. Disponible en: <https://docs.flutter.dev/ui/adaptive-responsive/general> [consulta: 30 de septiembre de 2026].

FLUTTER. Supported deployment platforms [en línea]. 2026b. Disponible en: <https://docs.flutter.dev/reference/supported-platforms> [consulta: 30 de septiembre de 2026].

FLUTTER. Testing Flutter apps [en línea]. 2026c. Disponible en: <https://docs.flutter.dev/testing/overview> [consulta: 2 de octubre de 2026].

FUDO. Funcionalidades: ¿Qué puedo hacer con Fudo? [en línea]. [s. f.]. Disponible en: <https://fu.do/es-ar/funcionalidades/> [consulta: 28 de septiembre de 2026].

GITHUB. GitHub Actions documentation [en línea]. [s. f.]. Disponible en: <https://docs.github.com/en/actions> [consulta: 2 de octubre de 2026].

GRAFANA LABS. Thresholds. Grafana k6 documentation [en línea]. [s. f.]. Disponible en: <https://grafana.com/docs/k6/latest/using-k6/thresholds/> [consulta: 2 de octubre de 2026].

HODGES, Jeff; JACKSON, Collin y BARTH, Adam. HTTP Strict Transport Security (HSTS). RFC 6797 [en línea]. IETF, 2012. DOI: 10.17487/RFC6797. Disponible en: <https://www.rfc-editor.org/info/rfc6797> [consulta: 2 de octubre de 2026].

JONES, Michael; BRADLEY, John y SAKIMURA, Nat. JSON Web Token (JWT). RFC 7519 [en línea]. IETF, 2015. DOI: 10.17487/RFC7519. Disponible en: <https://www.rfc-editor.org/info/rfc7519> [consulta: 2 de octubre de 2026].

KEYCLOAK. Securing applications and services with OpenID Connect [en línea]. [s. f.]. Disponible en: <https://www.keycloak.org/securing-apps/oidc-layers> [consulta: 30 de septiembre de 2026].

LOYVERSE. Sistema de pantalla de cocina para POS: Loyverse KDS [en línea]. [s. f.]. Disponible en: <https://loyverse.com/es/kitchen-display-system> [consulta: 28 de septiembre de 2026].

NODE.JS. Test runner. Node.js v24 documentation [en línea]. [s. f.]. Disponible en: <https://nodejs.org/docs/latest-v24.x/api/test.html> [consulta: 2 de octubre de 2026].

OPENAPI INITIATIVE. OpenAPI Specification, version 3.0.3 [en línea]. 2020. Disponible en: <https://spec.openapis.org/oas/v3.0.3> [consulta: 29 de septiembre de 2026].

OPEN SOURCE POINT OF SALE. Open Source Point of Sale [en línea]. [s. f.]. Disponible en: <https://github.com/opensourcepos/opensourcepos> [consulta: 28 de septiembre de 2026].

OPEN SOURCE POINT OF SALE. Releases [en línea]. [s. f.]. Disponible en: <https://github.com/opensourcepos/opensourcepos/releases> [consulta: 28 de septiembre de 2026].

ORACLE. KDS Basics [en línea]. [s. f.]. Disponible en: https://docs.oracle.com/en/industries/food-beverage/symphony/19.8/kdscu/c_kds_basics.htm [consulta: 28 de septiembre de 2026].

OWASP FOUNDATION. OWASP Top 10:2025 [en línea]. 2025. Disponible en: <https://owasp.org/Top10/2025/> [consulta: 2 de octubre de 2026].

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17.11 Documentation: BEGIN [en línea]. [s. f.a]. Disponible en: <https://www.postgresql.org/docs/17/sql-begin.html> [consulta: 30 de septiembre de 2026].

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17.11 Documentation: 5.5 Constraints [en línea]. [s. f.b]. Disponible en: <https://www.postgresql.org/docs/17/ddl-constraints.html> [consulta: 30 de septiembre de 2026].

SAKIMURA, Nat; BRADLEY, John y AGARWAL, Naveen. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636 [en línea]. IETF, 2015. DOI: 10.17487/RFC7636. Disponible en: <https://www.rfc-editor.org/info/rfc7636> [consulta: 29 de septiembre de 2026].

SAKIMURA, Nat; BRADLEY, John; JONES, Michael; DE MEDEIROS, Breno y MORTIMORE, Chuck. OpenID Connect Core 1.0 incorporating errata set 2 [en línea]. OpenID Foundation, 2023. Disponible en: https://openid.net/specs/openid-connect-core-1_0.html [consulta: 29 de septiembre de 2026].

SOCKET.IO. Introduction [en línea]. 2026. Disponible en: <https://socket.io/docs/v4/> [consulta: 30 de septiembre de 2026].

TAN, Tan-Hsu y CHANG, Ching-Su. Development and evaluation of an RFID-based e-restaurant system for customer-centric service. Expert Systems with Applications [en línea]. 2010, vol. 37, n.º 9, pp. 6482-6492. DOI: 10.1016/j.eswa.2010.02.137. Disponible en: <https://doi.org/10.1016/j.eswa.2010.02.137> [consulta: 28 de septiembre de 2026].

W3C. Content Security Policy Level 3 [en línea]. [s. f.]. Disponible en: <https://www.w3.org/TR/CSP3/> [consulta: 2 de octubre de 2026].

ANEXOS

Los seis anexos siguientes son obligatorios en esta área del conocimiento, conforme al punto 3.4 de los Lineamientos Técnicos.

Anexo A. Manual de usuario

Se completa en el E4.

Anexo B. Manual de instalación y despliegue

Se completa en el E4.

Anexo C. Enlace al repositorio de código

Repositorio público, sin necesidad de cuenta ni invitación:
<https://github.com/jorgeluisgutierrezrivera/max-pizzapp>

Para obtener una copia local: `git clone https://github.com/jorgeluisgutierrezrivera/max-pizzapp.git`. El archivo `README.md` describe el sistema, cómo correrlo en local y cómo ejecutar las pruebas. El historial de commits, fechados y en días distintos, registra el avance progresivo; la pestaña *Actions* muestra las corridas de la integración continua.

Anexo D. Dirección pública del sistema desplegado o archivo instalable

- **Aplicación web** (Recepción y Cocina): <https://maxpizzapp.tech>
- **Salud de la API**: <https://maxpizzapp.tech/api/v1/salud>
- **APK de Cocina** (Android 7 como mínimo configurado; validado en Honor 50 con Android 13): <https://github.com/jorgeluisgutierrezrivera/max-pizzapp/releases/latest/download/max-pizzapp-cocina.apk>

- **Usuarios de prueba:** `recepcion.demo` (rol Recepción) y `cocina.demo` (rol Cocina), con datos ficticios. Las contraseñas no figuran en este documento: se entregan en el campo de texto de la tarea.

Anexo E. Diagramas en tamaño legible

Se completa en el E4, con los diagramas de las figuras 2 a 5 en tamaño legible.

Anexo F. Documentación complementaria pertinente

La documentación complementaria está en el repositorio:

- el contrato de la API en OpenAPI 3.0.3: `docs/api/openapi.yaml`;
- el plan de pruebas y la tabla de casos: `docs/pruebas/README.md`, con los informes de las pruebas automatizadas y de las mediciones en `docs/pruebas/reportes/`;
- el plan de cada tarjeta, con la evidencia de sus pruebas y los commits que la cerraron: `docs/planes/`;
- la especificación de desarrollo: `docs/BRIEF.md`.

Matriz de requisitos

Anexo F · Versión del E2 revisada para el E3, derivada de la matriz de la cápsula C2

Contiene los dos actores, los 14 requisitos funcionales con sus 28 criterios de aceptación, los cinco requisitos no funcionales con métrica, las siete capacidades declaradas fuera de alcance y el resumen MoSCoW.

El archivo editable se entrega además en formato XLSX.

A. Actores del sistema

ID	Actor	Descripción	Qué puede hacer	Qué no puede hacer
A-01	Recepción	Personal que atiende el mostrador: recibe al cliente, toma el pedido y lo entrega cuando está listo. Opera desde tableta o equipo de escritorio.	Iniciar sesión · Consultar la carta · Registrar un pedido y su observación · Ver el estado de los pedidos activos en vivo · Agregar productos a un pedido ya enviado · Cancelar un pedido mientras siga pendiente · Marcar como entregado un pedido listo · Marcar un producto como agotado	Cambiar el estado a en preparación o listo (es competencia de cocina) · Marcar entregado un pedido que no está listo · Crear, editar o eliminar productos ni modificar precios · Eliminar pedidos ni alterar el historial
A-02	Cocina	Personal de preparación. Opera desde tableta o monitor fijo en cocina, de pie y sin teclado.	Iniciar sesión · Ver la cola de pedidos entrantes por hora de llegada, en vivo · Consultar el detalle y la observación · Pasar un pedido a en preparación · Marcar un pedido como listo · Marcar un producto como agotado	Registrar pedidos · Marcar pedidos como entregados · Cancelar pedidos · Crear, editar o eliminar productos ni modificar precios · Consultar datos de contacto del cliente

Tabla A.1 Actores. Fuente: elaboración propia.

B. Requisitos funcionales y criterios de aceptación

ID	Requisito funcional (historia de usuario)	Criterio de aceptación (camino feliz)	Criterio de aceptación (camino de error)	MoSCoW	Actor	Iteración
RF-01	Como usuario del sistema, quiero iniciar sesión con mis credenciales, para acceder a las funciones de mi rol.	CA-01.1 Dado un usuario con credenciales válidas, cuando las introduce en el proveedor de identidad, entonces el sistema recibe su token, lo valida y lo dirige a la pantalla correspondiente a su rol.	CA-01.2 Dado un usuario con credenciales inválidas, cuando intenta acceder, entonces el proveedor rechaza el acceso sin emitir token y el usuario permanece fuera del sistema, que no muestra ninguna pantalla de su rol.	Must	Ambos	Semana 2
RF-02	Como recepción, quiero registrar un pedido seleccionando productos de la carta, para enviarlo a cocina.	CA-02.1 Dada una carta con productos disponibles, cuando la recepción selecciona productos y confirma, entonces el sistema crea el pedido en estado pendiente y lo envía a cocina.	CA-02.2 Dado un pedido sin productos, cuando la recepción intenta confirmarlo, entonces el sistema no lo crea y muestra: Agregue al menos un producto.	Must	Recepción	Semana 2
RF-03	Como recepción, quiero ver el estado de cada pedido actualizado en tiempo real, para conocer su avance sin consultar a cocina.	CA-03.1 Dado un pedido activo, cuando cocina cambia su estado, entonces la pantalla de recepción refleja el nuevo estado en menos de 2 segundos sin recargar.	CA-03.2 Dada una caída del canal en tiempo real, cuando se pierde la conexión, entonces el sistema muestra un aviso y permite recargar para ver el estado actual.	Must	Recepción	Semana 3
RF-04	Como recepción, quiero recibir un aviso cuando un pedido pase a listo, para entregarlo de inmediato.	CA-04.1 Dado un pedido en preparación, cuando cocina lo marca listo, entonces recepción recibe un aviso visible y el pedido aparece destacado.	CA-04.2 Dado que no hay pedidos listos, cuando la recepción abre su pantalla, entonces se muestra el estado vacío sin avisos pendientes.	Must	Recepción	Semana 3
RF-05	Como recepción, quiero marcar un pedido como entregado, para cerrar su atención.	CA-05.1 Dado un pedido en estado listo, cuando la recepción lo marca como entregado, entonces el sistema registra la entrega y lo retira de la lista activa.	CA-05.2 Dado un pedido que aún no está listo, cuando la recepción intenta entregarlo, entonces el sistema lo impide y muestra: El pedido aún no está listo.	Must	Recepción	Semana 3
RF-06	Como cocina, quiero ver en tiempo real los pedidos entrantes ordenados por llegada, para prepararlos en orden.	CA-06.1 Dado un nuevo pedido confirmado por recepción, cuando se crea, entonces aparece en la pantalla de cocina en menos de 2 segundos, al final de la cola.	CA-06.2 Dado que no hay pedidos pendientes, cuando cocina abre su pantalla, entonces se muestra el estado vacío.	Must	Cocina	Semana 2
RF-07	Como cocina, quiero cambiar el estado de un pedido (en preparación / listo), para informar el avance a recepción.	CA-07.1 Dado un pedido pendiente, cuando cocina lo pasa a en preparación y luego a listo, entonces el sistema guarda cada cambio y lo propaga a recepción en tiempo real.	CA-07.2 Dado un pedido ya entregado, cuando cocina intenta cambiar su estado, entonces el sistema lo impide por tratarse de un estado final.	Must	Cocina	Semana 3
RF-08	Como usuario del sistema, quiero cerrar sesión y que otro compañero inicie la suya en el mismo dispositivo, para que cada acción quede	CA-08.1 Dado un usuario con sesión iniciada, cuando cierra sesión, entonces el sistema descarta el token, vuelve a la	CA-08.2 Dado un token expirado, cuando el usuario intenta una operación, entonces el sistema la rechaza y lo devuelve a la pantalla de acceso.	Should	Ambos	Semana 4

ID	Requisito funcional (historia de usuario)	Criterio de aceptación (camino feliz)	Criterio de aceptación (camino de error)	MoSCoW	Actor	Iteración
	registrada a nombre de quien la ejecuta.	pantalla de acceso y el siguiente usuario inicia su propia sesión.				
RF-09	Como recepción, quiero cancelar un pedido mientras cocina no lo haya empezado, para corregir errores de toma sin desperdiciar lo que ya se preparó.	CA-09.1 Dado un pedido en pendiente, cuando la recepción lo cancela indicando un motivo, entonces el sistema lo marca cancelado y lo notifica a cocina en tiempo real.	CA-09.2 Dado un pedido en en preparación, listo o entregado, cuando se intenta cancelar, entonces el sistema lo impide.	Should	Recepción	Semana 3
RF-10	Como cocina, quiero ver cuánto tiempo lleva esperando cada pedido de la cola, para priorizar los más antiguos.	CA-10.1 Dado un pedido en la cola, cuando transcurre el tiempo, entonces la tarjeta muestra los minutos transcurridos desde su registro, actualizados sin recargar.	CA-10.2 Dado un pedido activo cuyo tiempo transcurrido desde su registro supera los 15 minutos —el extremo superior de la duración normal de un pedido, según el apartado 1.2—, cuando se muestra en la cola de cocina, entonces se destaca con la etiqueta «demorado», además del color.	Could	Cocina	Semana 4
RF-11	Como recepción, quiero agregar una observación a un pedido, para comunicar indicaciones a cocina.	CA-11.1 Dado un pedido en registro, cuando la recepción agrega una nota, entonces la nota se muestra en cocina junto al pedido.	CA-11.2 Dada una nota que excede el máximo permitido, cuando se intenta guardar, entonces el sistema la rechaza e indica el límite.	Should	Recepción	Semana 3
RF-12	Como recepción, quiero visualizar un comprobante del pedido, para mostrarlo al cliente.	CA-12.1 Dado un pedido registrado, cuando la recepción abre su comprobante, entonces el sistema muestra productos, cantidades, total y número de pedido.	CA-12.2 Dado un pedido inexistente, cuando se solicita su comprobante, entonces el sistema muestra: Pedido no encontrado.	Could	Recepción	Semana 4
RF-13	Como cocina o recepción, quiero marcar un producto como agotado, para que deje de ofrecerse en la carta y no se tomen pedidos que no se pueden preparar.	CA-13.1 Dado un producto disponible, cuando cocina o recepción lo marca como agotado, entonces el sistema lo deja de mostrar como seleccionable en la carta y el cambio llega a las pantallas conectadas en menos de 2 segundos.	CA-13.2 Dado un producto agotado que ya figura en un pedido en curso, cuando se marca como agotado, entonces el sistema conserva ese pedido sin alterarlo y solo impide nuevas selecciones del producto.	Should	Ambos	Semana 3
RF-14	Como recepción, quiero agregar productos a un pedido ya enviado, para que el cliente que pide algo más después no tenga que hacer un pedido aparte.	CA-14.1 Dado un pedido pendiente o en preparación, cuando la recepción le agrega una bebida o una pizza, entonces el sistema suma los productos y el total, y cocina los ve marcados como agregados en menos de 2 segundos.	CA-14.2 Dado un pedido listo, entregado o cancelado, cuando se intenta agregarle una pizza, entonces el sistema lo impide; a un pedido listo solo se le pueden agregar bebidas.	Should	Recepción	Semana 2

Tabla A.2 Requisitos funcionales. Fuente: elaboración propia.

C. Requisitos no funcionales y su métrica

ID	Tipo	Enunciado	Métrica de verificación
RNF-01	Rendimiento	Los cambios de estado deben reflejarse en las pantallas conectadas sin intervención del usuario, y el listado principal debe cargar con fluidez en la carga habitual del local	Propagación de un cambio de estado en menos de 2 segundos , medida con un script propio sobre socket.io-client que registra el instante de la petición y el de la recepción del evento en 30 repeticiones y reporta el peor caso; carga del listado principal en menos de 2 segundos con 50 pedidos activos y 5 usuarios concurrentes , medida con k6 sobre GET /api/v1/pedidos y reportando el percentil 95
RNF-02	Seguridad	El acceso se autentica contra el proveedor de identidad, las comunicaciones viajan cifradas y la autorización por rol se comprueba en el servidor	Autenticación OpenID Connect con Keycloak 26.7; token JWT con expiración de 60 minutos; HTTPS en todo el tráfico público; contraseñas con hash gestionadas por el proveedor; el 100 % de las rutas protegidas valida el token y los permisos que exige la ruta; las operaciones que cambian el estado de un pedido comprueban además que la transición solicitada sea válida
RNF-03	Usabilidad	La tarea principal debe completarse en pocos pasos y toda vista con datos debe informar su situación al usuario	Registrar un pedido en 3 pasos o menos; 4 estados implementados (cargando, con datos, vacío y error) en todas las vistas que consumen datos
RNF-04	Compatibilidad	El sistema debe funcionar en los navegadores del local, tanto en equipo de escritorio como en tableta	Chrome y Edge, últimas 2 versiones; interfaz adaptable verificada a 1366 px (escritorio) y 768 px (tableta), sin desplazamiento horizontal
RNF-05	Disponibilidad	El sistema debe estar disponible durante el horario de atención y degradarse de forma controlada si cae el canal en tiempo real	Disponible durante el horario de atención del local, de 18:00 a 23:30 , verificado con un sondeo automático de GET /api/v1/salud cada 5 minutos durante un periodo de observación de al menos 7 días , declarando en el apartado 2.8 las fechas de inicio y fin efectivamente medidas, con ≥ 99 % de respuestas 200 dentro de esa franja; ante caída del canal, aviso visible al usuario en menos de 10 segundos y opción de recarga manual para consultar el estado actual

Tabla A.3 Requisitos no funcionales. Fuente: elaboración propia.

D. Capacidades declaradas fuera de alcance

Capacidad excluida	MoSCoW	Justificación de la exclusión
Operación multi-sede: sucursales de Villa Avaroa y Tabladita, con aislamiento de datos por local	Won't	El proyecto se acota a un local piloto. El aislamiento de datos por sede añadiría complejidad sin aportar a la sincronización en tiempo real, que es el eje del trabajo. Queda como trabajo futuro en el apartado 3.2.
Control de inventario: existencias por día y local, descuento de stock al confirmar el pedido y bloqueo de venta sin stock	Won't	Formaba parte del planteamiento original y se recortó en la tutoría T1. Exige un dominio propio —existencias, mermas, reposición— que no interviene en la coordinación entre recepción y cocina, que es el eje del trabajo. La necesidad inmediata que cubría —no ofrecer lo que no se puede preparar— se resuelve con RF-13.
Control de caja y ventas: cobros, arqueos y reportes de venta	Won't	Es un dominio propio, con sus reglas y su normativa, que excede el plazo de cuatro semanas del módulo y no interviene en la coordinación entre recepción y cocina.
Promociones y combos con condiciones: descuentos por día u horario, 2x1 y porcentajes	Won't	Una promoción no es un dato sino una regla de precio, con fechas y condiciones, y exigiría un motor que la aplique justo sobre el cálculo del precio, la parte más delicada del pedido. No figura en los requisitos y excede el plazo del módulo. Un combo de precio fijo puede cargarse más adelante como un producto de la carta, sin código nuevo. Queda como trabajo futuro en el apartado 3.2
Módulo de despacho como rol independiente	Won't	En el local piloto la entrega la realiza la propia recepción en mostrador. Crear un tercer rol multiplicaría pantallas, reglas de autorización y casos de prueba sin corresponder a la operación real.
Gestión de la carta (alta, edición y baja de productos) por un rol administrador	Could	Requiere un tercer rol diferenciado, por encima del máximo admitido. Durante el piloto la carta se carga en la base de datos con los datos iniciales. Queda como mejora recomendada en el apartado 3.2.
Consulta del historial del día por un rol administrador	Could	Los datos quedan registrados en la entidad historial_estado, de modo que la capacidad puede añadirse después sin cambiar el modelo; se excluye de esta versión por la misma limitación de roles.

Tabla A.4 Fuera de alcance. Fuente: elaboración propia.

E. Resumen MoSCoW

Categoría	Requisitos	Cantidad	Porcentaje
Must have	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07	7	50 %
Should have	RF-08, RF-09, RF-11, RF-13, RF-14	5	36 %
Could have	RF-10, RF-12	2	14 %
Total		14	100 %
Límite del módulo	Los Must have no pueden superar el 60 % del backlog		Cumple: 50 %
Actores	Dos roles diferenciados: A-01 Recepción y A-02 Cocina		
Fuente	Elaboración propia. Max Pizzapp — Jorge Luis Gutierrez Rivera, UAJMS 2026		
Postulante	Jorge Luis Gutierrez Rivera		
Correo	e73700@uajms.edu.bo		
Proyecto	Max Pizzapp — sistema web de gestión de pedidos con sincronización en tiempo real para la pizzería Max's Pizzas		
Entrega	Revisada para el E3 · 3 de octubre de 2026 (versión del E2 del 26 de septiembre) (entregada con la cápsula C2 el 19 de septiembre)		

Tabla A.5 Resumen. Fuente: elaboración propia.